



UNIVERSITÀ
di **VERONA**

Deep Learning

GENERATIVE MODELS

Cigdem Beyan
A. Y. 2025/2026

TODAY



UNIVERSITÀ
di **VERONA**

- Autoencoders
- Variational Autoencoders
- Generative Adversarial Networks
- Pros, cons, application areas, trends...

SUPERVISED LEARNING

Data: $(\mathbf{x}, y)$, $\mathbf{x}$ is data, y is label

Goal: Learn a *function* to map $f: \mathbf{x} \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.



→ Cat

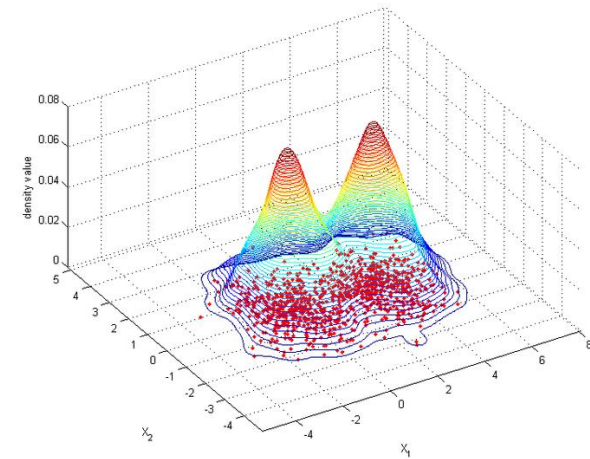
Classification

UNSUPERVISED LEARNING

Data: $\mathbf{x}$, Just data, no labels!

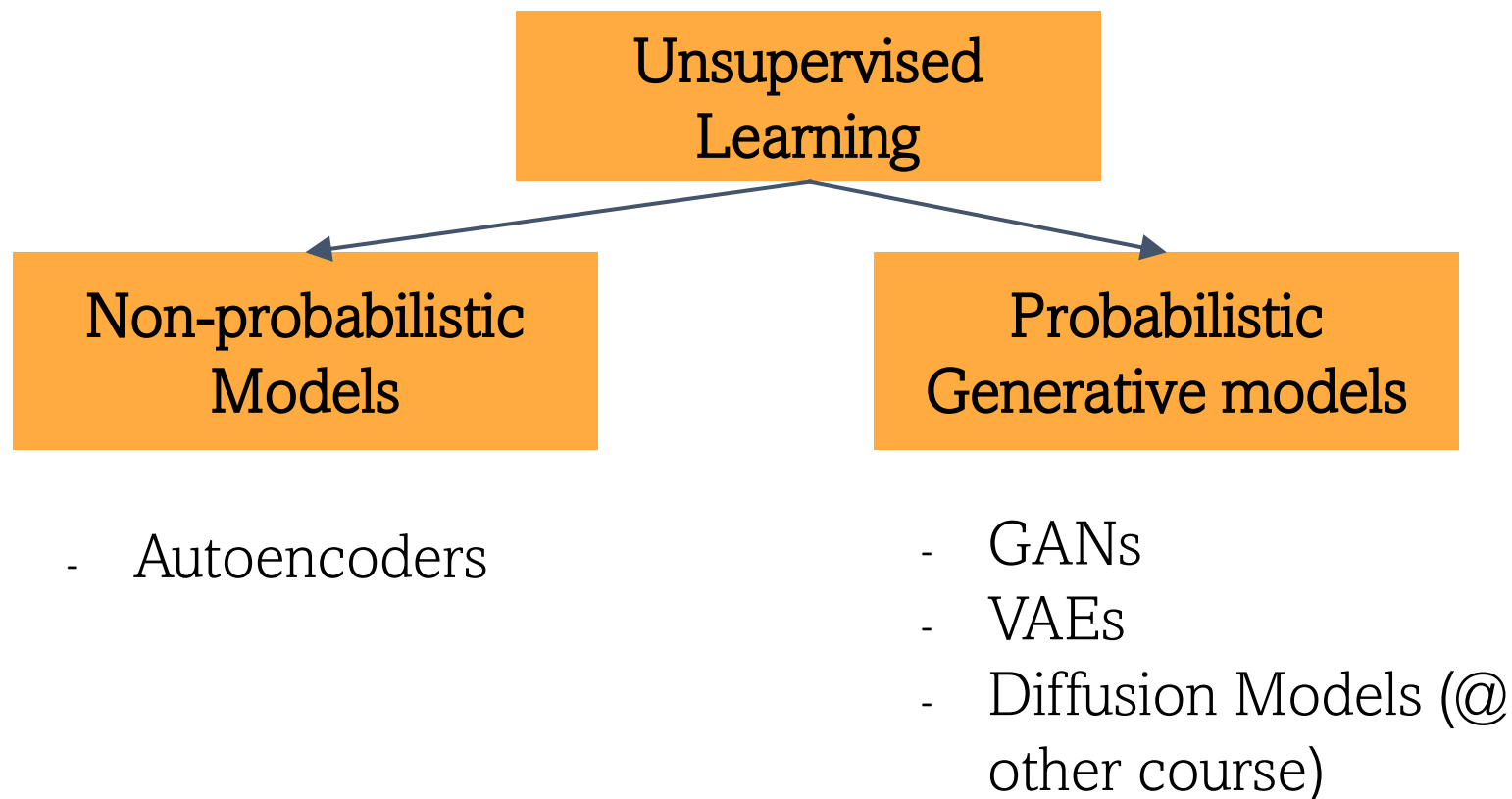
Goal: Learn some underlying hidden structure of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.



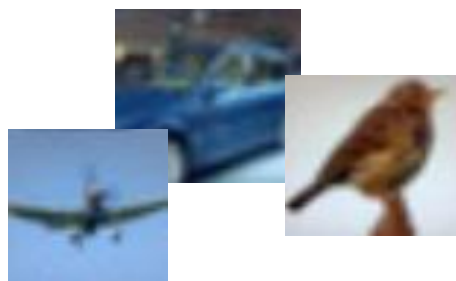
Density Estimation

UNSUPERVISED LEARNING: DL TAXONOMY



GENERATIVE MODELS

Given training data, generate new samples from same distribution



Training data $\sim p_{\text{data}}(\mathbf{x})$



Generated samples $\sim p_{\text{model}}(\mathbf{x})$

Density estimation

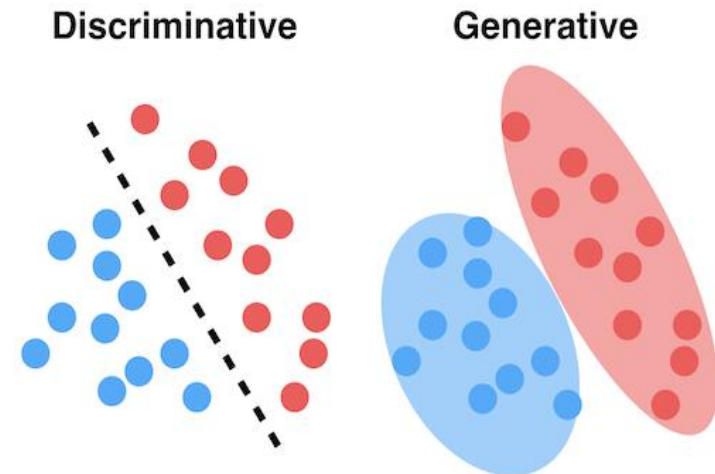
Want to learn $p_{\text{model}}(\mathbf{x})$ similar to $p_{\text{data}}(\mathbf{x})$

GENERATIVE MODELS

Given a Training set X with the associated labels Y :

Discriminative models $p(Y|X)$

Generative models $p(X)$

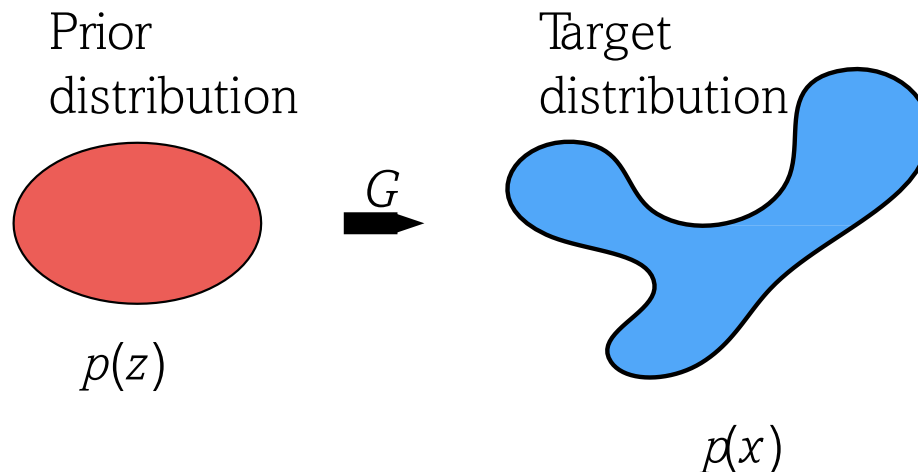


GENERATIVE MODELS

Goal: Take as input unlabeled training samples from some distribution and learn a model that represents that distribution

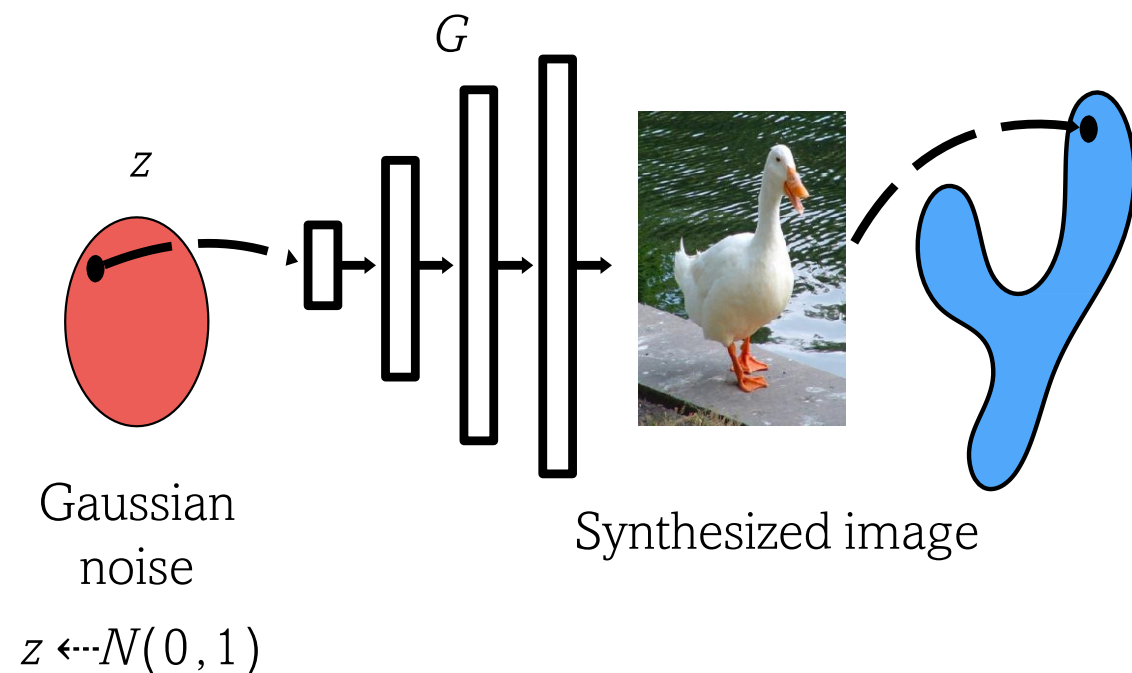
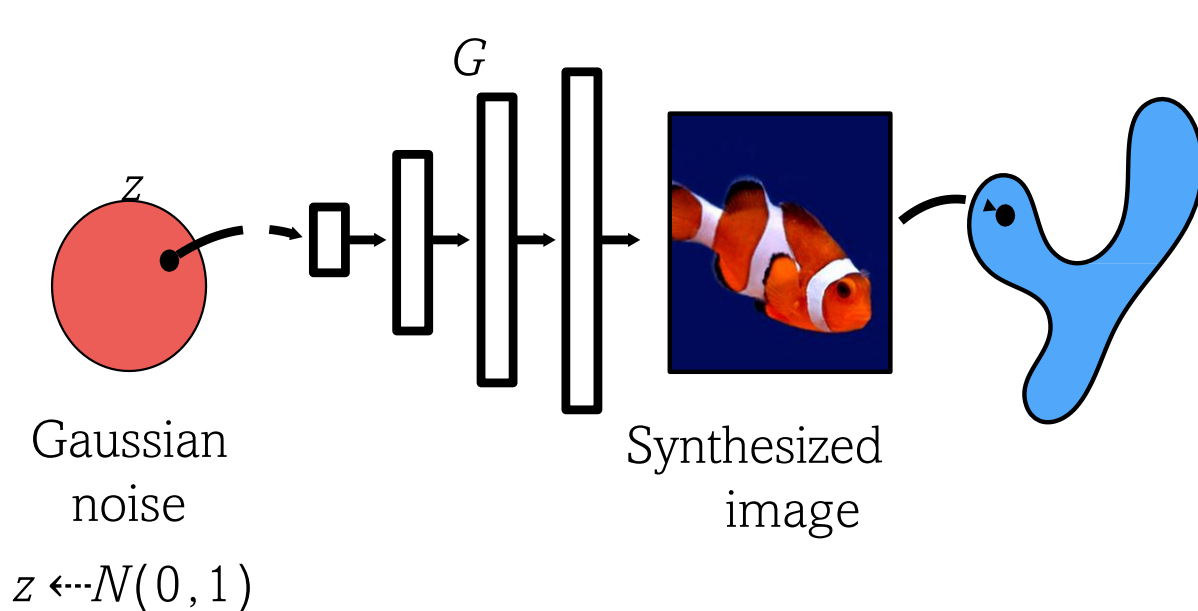
- density estimation
- learn appropriate representations or embeddings
- generate new data (with same or different distribution)

Deep generative models are distribution transformers



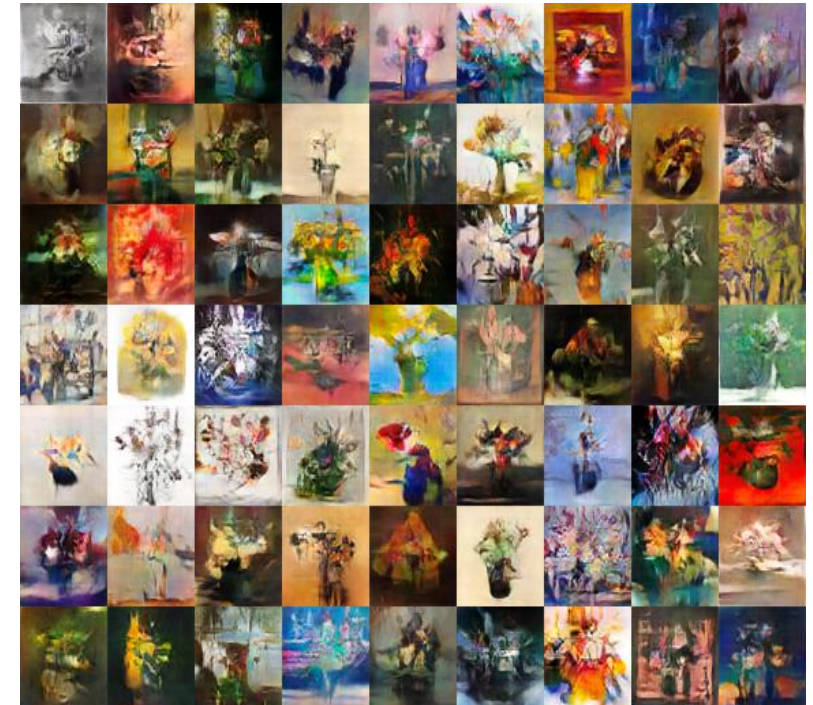
GENERATIVE MODELS

Deep generative models are distribution transformers



WHY GENERATIVE MODELS

Realistic samples for artwork, super-resolution, colorization, etc.



WHY GENERATIVE MODELS

Simulation and planning (e.g. reinforcement learning, transform synthetic data into realistic ones)



(a) GTA5



(b) GTA5 → Cityscapes

AUTOENCODERS



UNIVERSITÀ
di **VERONA**

Biol. Cybern. 59, 291–294 (1988)

Biological
Cybernetics
© Springer-Verlag 1988

Auto-Association by Multilayer Perceptrons and Singular Value Decomposition

H. Bourlard and Y. Kamp

Philips Research Laboratory, Avenue Van Becelaere 2, Box 8, B-1170 Brussels, Belgium

Abstract. The multilayer perceptron, when working in auto-association mode, is sometimes considered as an interesting candidate to perform data compression or dimensionality reduction of the feature space in information processing applications. The present paper shows that, for auto-association, the nonlinearities of the hidden units are useless and that the optimal parameter values can be derived directly by purely linear techniques relying on singular value decomposition and low rank matrix approximation, similar in spirit to the well-known Karhunen-Loève transform. This approach appears thus as an efficient alternative to the general error back-propagation algorithm commonly used for training multilayer perceptrons. Moreover, it also gives a clear interpretation of the rôle of the different parameters.

1 Introduction

Multilayer perceptrons (MLP) form a class of neural networks in which the nonlinear computing elements are arranged in a feed-forward layered structure. Since their emergence, they have been quite successfully used for a wide variety of tasks such as information processing, coding and pattern recognition, see e.g. Rumelhart and McClelland (1986) or Lippmann (1987). One of these applications consists in using the MLP to reduce the dimension of the feature space, an idea suggested by Rumelhart and McClelland (1986) and actually implemented by Elman and Zipser (1988) and Harrison (1987) in speech recognition and by Cottrell et al. (1988) for image compression. For this particular mode of operation, known as auto-association, identity mapping or encoding, the target output pattern is identical to the input pattern and, as a consequence, the output layer does generally not contain any nonlinear function, at least for real valued inputs. One can, of course, still apply the usual error

back-propagation (EBP) algorithm as described by Rumelhart et al. (1986) to obtain the optimal parameter values for this case. However, the purpose of this paper is to show that, for auto-association with linear output units, the optimal weight values can be derived by standard linear algebra, consisting essentially in singular value decomposition (SVD) and making thus the nonlinear functions at the hidden layer completely unnecessary. The advantages are then obvious: the solution is obtained explicitly in terms of the training data, whereas the EBP generally used for the training of MLP proceeds iteratively and may well miss the optimum since it relies on a gradient technique and thus can get trapped in local minima. The analysis presented below offers the additional benefit that the optimal parameters are given a meaningful interpretation in terms of reconstruction of the average value and covariance of the input vectors.

2 MLP for Auto-Association

Figure 1 represents a general MLP with one hidden layer. It consists in an input layer containing n_i units and two layers with computational units: the hidden layer having p units and the output layer with n_o units. In the standard MLP, the output of each hidden and output unit is determined by forming a weighted sum of the unit values in the preceding layer and then passing this result through a sigmoidal function, e.g.

$$F(x) = 1/(1 + e^{-x}). \quad (1)$$

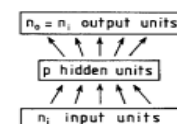
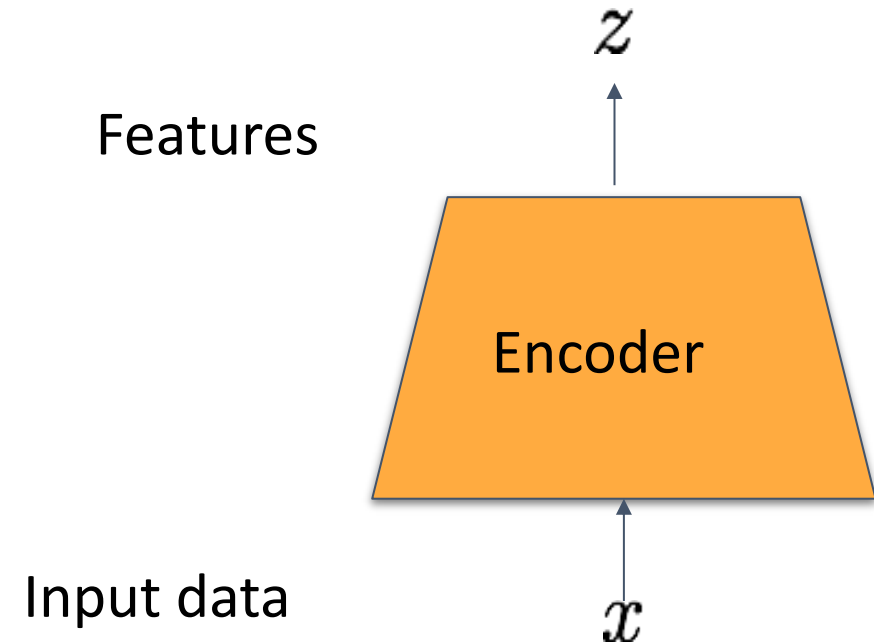


Fig. 1. MLP with one hidden layer for auto-association

AUTOENCODERS: ARCHITECTURE

- Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data.
- $\mathbf{z}$ is usually smaller than $\mathbf{x}$ (dimensionality reduction), as we want features to capture meaningful factors of variation in data.



AUTOENCODERS: ARCHITECTURE

- Train the model in order to learn features such that they can be used to reconstruct the original data.
- **Autoencoding:** encoding itself.
- An efficient way to learn features
 - Still need a loss function → for implicit supervision

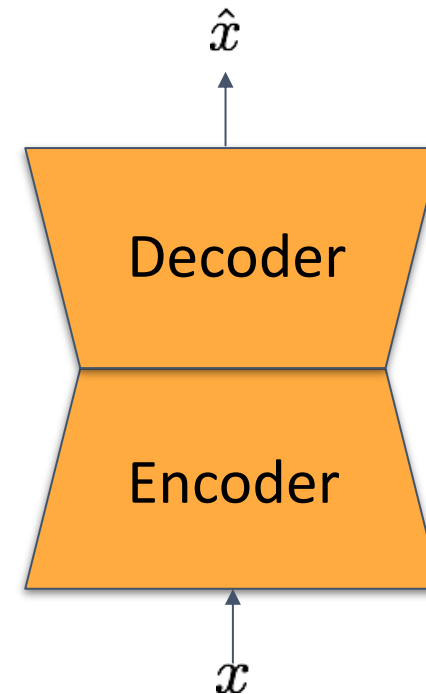
L2 Loss function:

$$\|x - \hat{x}\|^2$$

Reconstructed
Input

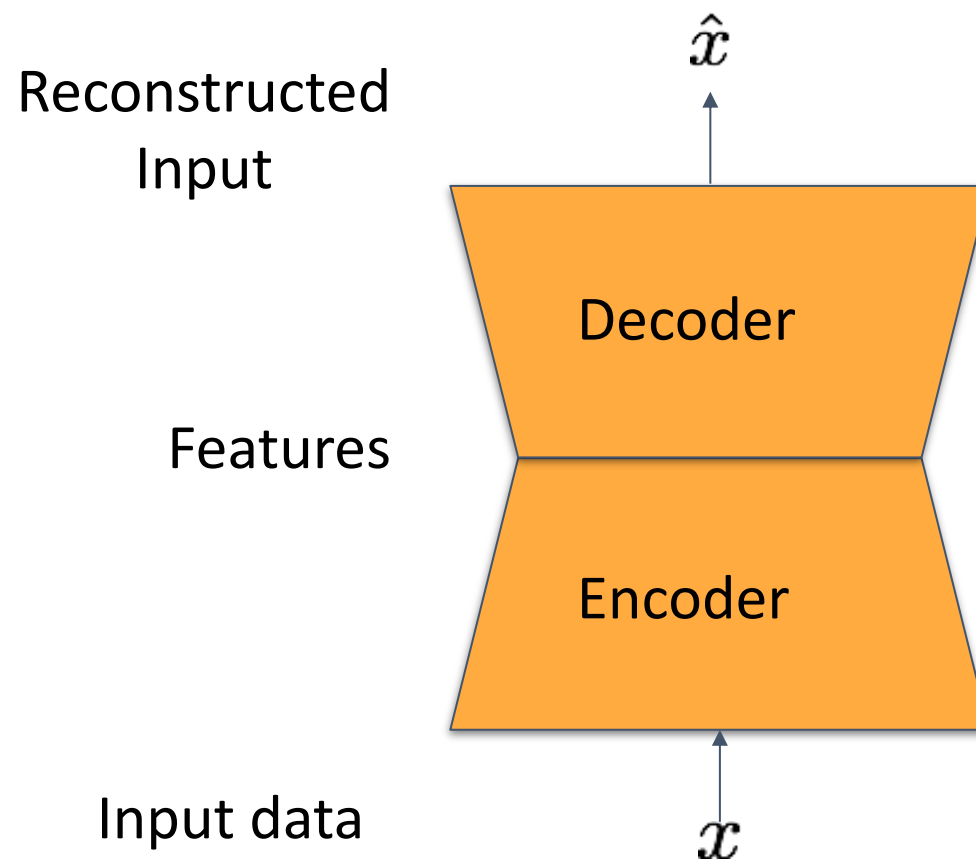
Features

Input data



AUTOENCODERS: IMPLEMENTATION

- **Encoder:**
 - $h=f(x)$: it describes the lower-dimensional code to represent the input.
 - Originally: Linear + nonlinearity (sigmoid)
 - Later: Deep, fully-connected
 - Much later: CNN, ReLU (for images)
- **Decoder:** mirroring structure
 - $x' = g(h)$, that produces the approximate reconstruction.



AUTOENCODERS

- Traditionally (1987... 1994), they were used for **dimensionality reduction** and **feature learning**.
 - If we design an autoencoder composed of linear layers, and if we do not use any non-linear activations and use the L2 loss function (in other words, MSE)

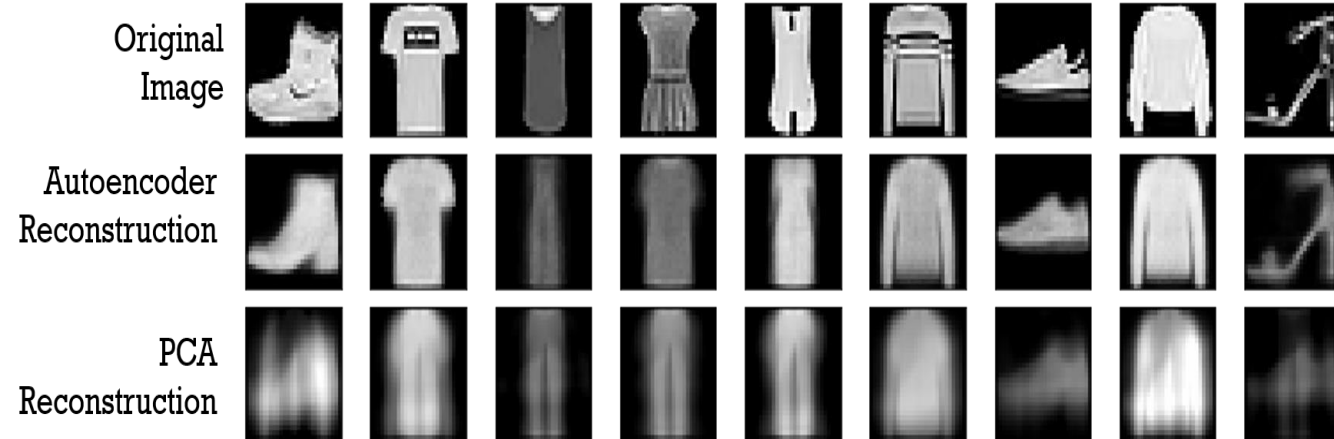
$$L(x, x') = ||x - x'||_2^2 = \sum_i (x_i - x'_i)^2$$

then, we obtain something very similar to PCA.

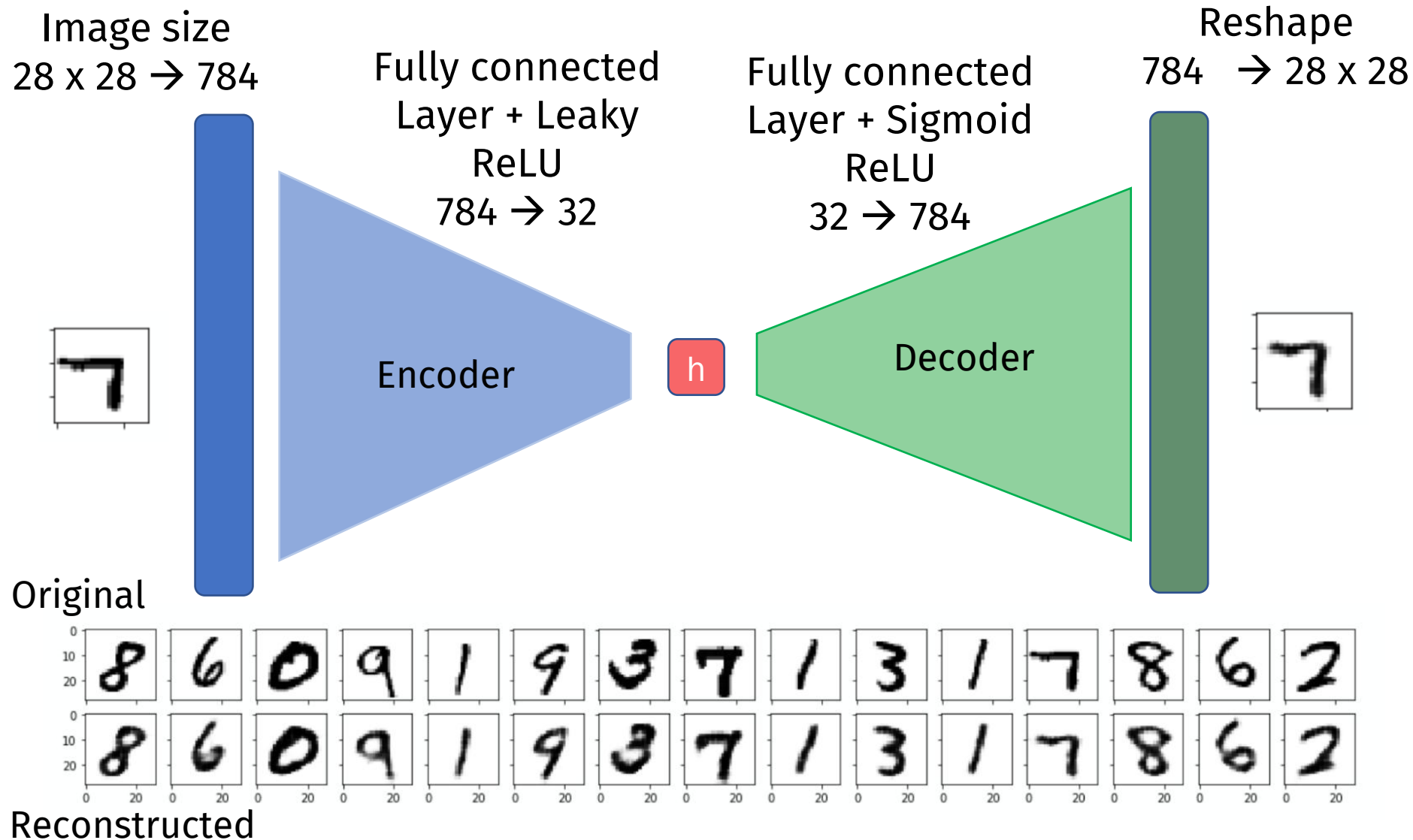
- More recently, they have been applied as **generative models**.

AUTOENCODERS

- Compare PCA/SVD (Singular value Decomposition)
 - PCA produces a smaller set of vectors.
 - Approximate the input vectors via linear combination.
 - Very efficient for certain applications.
- Autoencoder
 - Can learn nonlinear dependencies.
 - Can use convolutional layers.
 - Can be used for transfer learning.



AUTOENCODERS: AN EXAMPLE



DIMENSIONALITY OF THE LATENT SPACE

Reconstruction quality

2D Latent Space

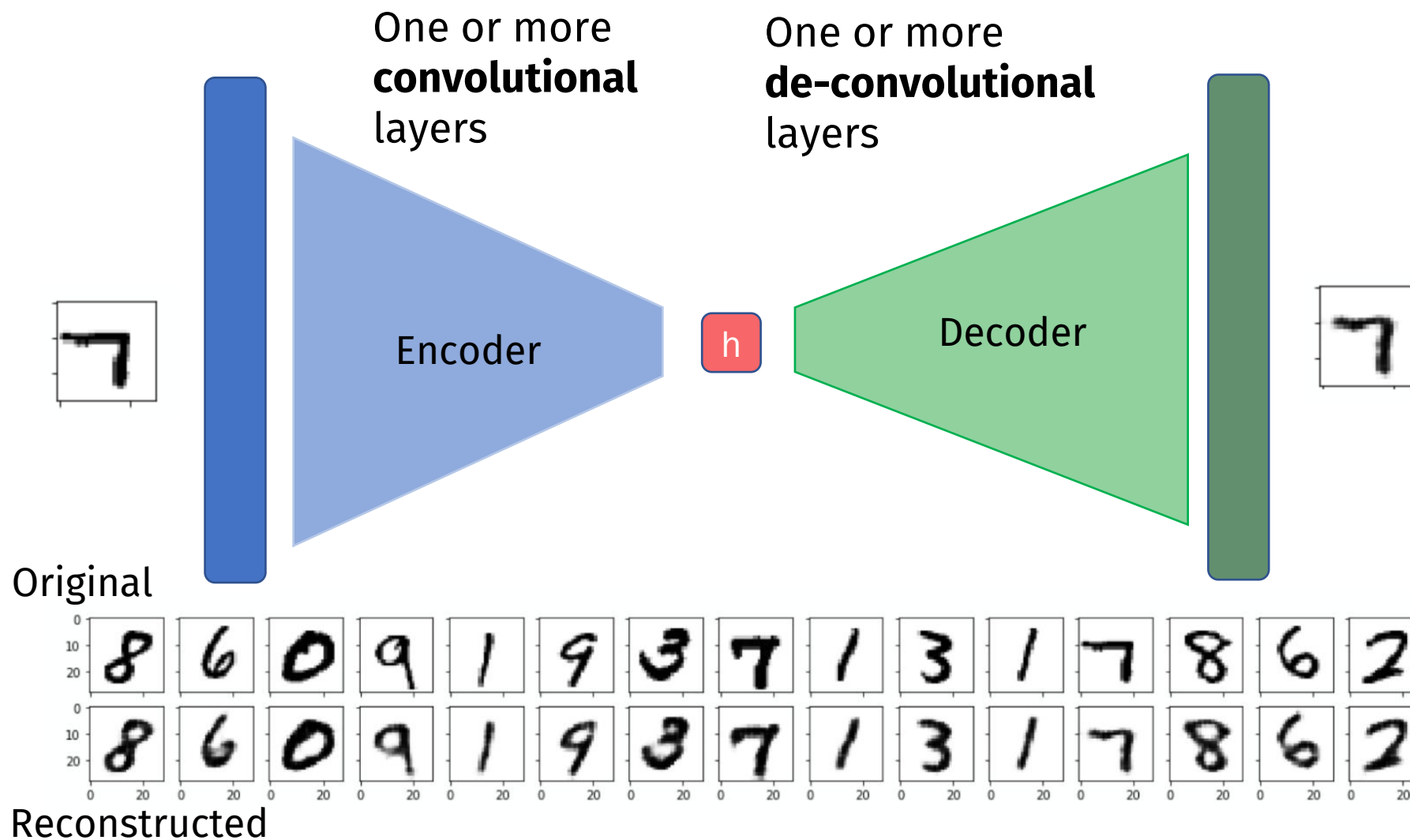


5D Latent Space



- Autoencoding compresses!

CONVOLUTIONAL AUTOENCODERS





REGULARIZED AUTOENCODERS

- You might have high capacity when the model has equal or higher dimension than the input. In the latter case, it is called an **overcomplete autoencoder**.
- **Regularized autoencoder** provides the ability to train an architecture, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of the distribution.
- **Idea:** Regularized autoencoders use a loss function that encourages the model to have certain properties (as sparsity, for instance).

REGULARIZED AUTOENCODERS

- An example of a regularized autoencoder is the **sparse autoencoder**, when you apply a sparsity penalty on the code layer, so that the loss becomes:

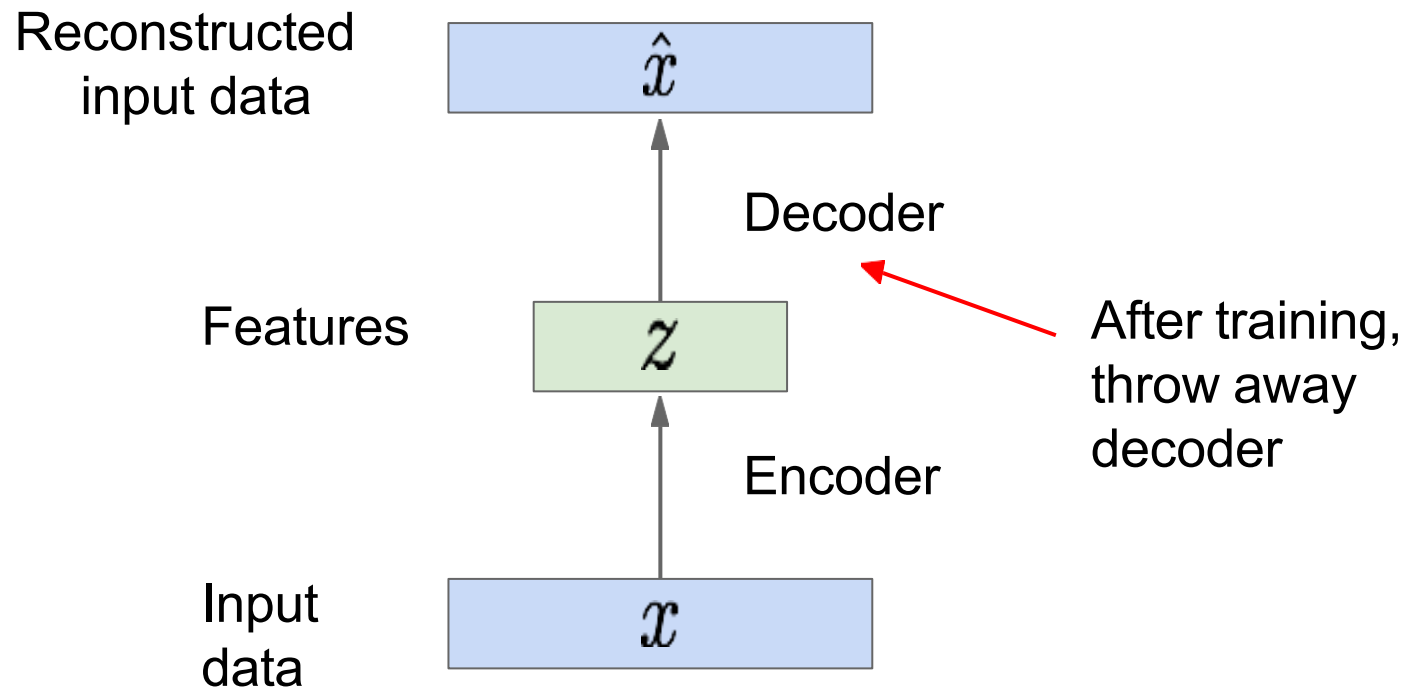
$$L(x, g(f(x))) + \lambda ||h||_1$$

- An alternative is to penalize the derivatives, forcing to learn of a function that does not change much when the input (x) slightly changes:

$$L(x, g(f(x))) + \lambda \sum_i ||\nabla_x(h_i)||^2$$

AUTOENCODERS

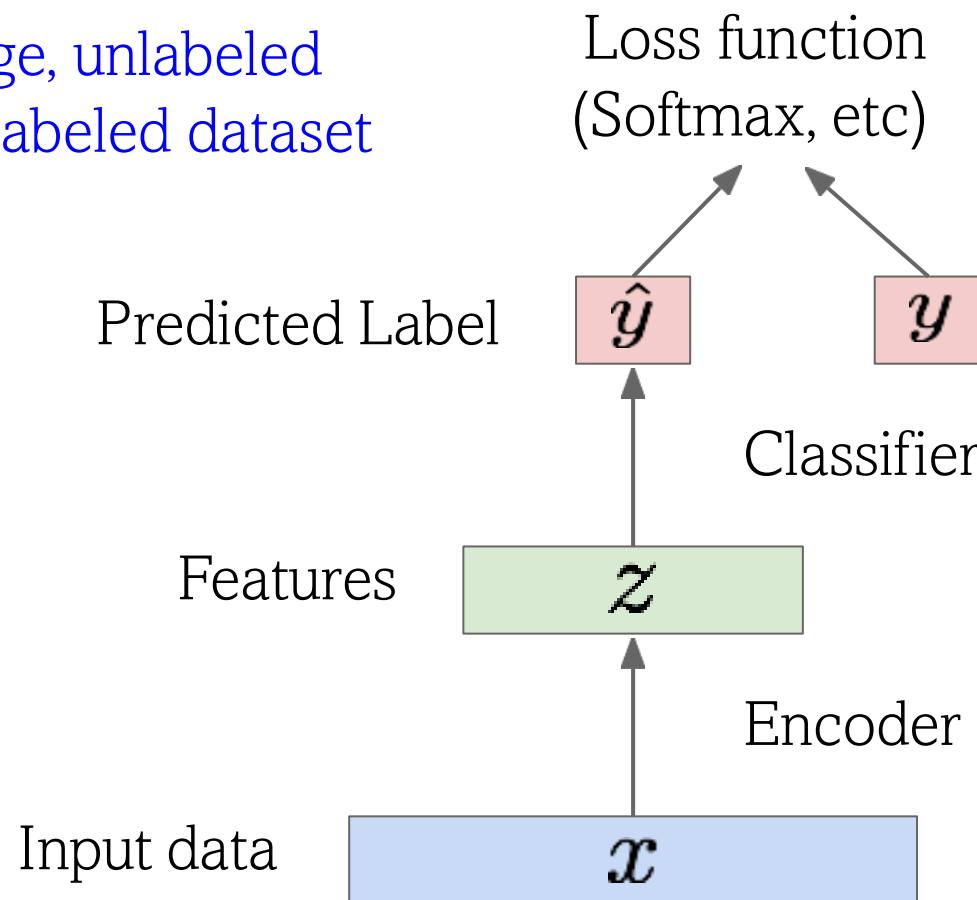
- Can we use the learned features (also called latent space) of the autoencoder for a discrimination task?
 - Yes (those features are typically very powerful, more compact representations).
- After training, throw away the decoder; the encoder can be used to initialize a supervised model.



AUTOENCODERS

Transfer from large, unlabeled
dataset to small, labeled dataset

Encoder can be
used to initialize a
supervised model



Fine-tune
encoder
jointly with
classifier

bird plane
dog deer truck

Train for final task
(sometimes with
small data)





AUTOENCODERS: SUMMARY

- Dimensionality reduction technique based on deep networks.
- Very flexible for different types of data.

Research threads in past years:

- Denoising Autoencoders (next!)
- Sparse Autoencoders (briefly mentioned before)
- Variational Autoencoders (next!)

DENOISING AUTOENCODERS



UNIVERSITÀ
di **VERONA**

Extracting and Composing Robust Features with Denoising Autoencoders

Pascal Vincent
Hugo Larochelle
Yoshua Bengio
Pierre-Antoine Manzagol

Université de Montréal, Dept. IRO, CP 6128, Succ. Centre-Ville, Montréal, Québec, H3C 3J7, Canada

VINCENTP@IRO.UMONTREAL.CA
LAROCHEH@IRO.UMONTREAL.CA
BENGIOY@IRO.UMONTREAL.CA
MANZAGOP@IRO.UMONTREAL.CA

Abstract

Previous work has shown that the difficulties in learning deep generative or discriminative models can be overcome by an initial unsupervised learning step that maps inputs to useful intermediate representations. We introduce and motivate a new training principle for unsupervised learning of a representation based on the idea of making the learned representations robust to partial corruption of the input pattern. This approach can be used to train autoencoders, and these denoising autoencoders can be stacked to initialize deep architectures. The algorithm can be motivated from a manifold learning and information theoretic perspective or from a generative model perspective. Comparative experiments clearly show the surprising advantage of corrupting the input of autoencoders on a pattern classification benchmark suite.

1. Introduction

Recent theoretical studies indicate that deep architectures (Bengio & Le Cun, 2007; Bengio, 2007) may be needed to *efficiently* model complex distributions and achieve better generalization performance on challenging recognition tasks. The belief that additional levels of functional composition will yield increased representational and modeling power is not new (McClelland et al., 1986; Hinton, 1989; Utgoff & Straczuzi, 2002). However, in practice, learning in deep architectures has proven to be difficult. One needs only

to ponder the difficult problem of inference in deep directed graphical models, due to “explaining away”. Also looking back at the history of multi-layer neural networks, their difficult optimization (Bengio et al., 2007; Bengio, 2007) has long prevented reaping the expected benefits of going beyond one or two hidden layers. However this situation has recently changed with the successful approach of (Hinton et al., 2006; Hinton & Salakhutdinov, 2006; Bengio et al., 2007; Ranzato et al., 2007; Lee et al., 2008) for training Deep Belief Networks and stacked autoencoders.

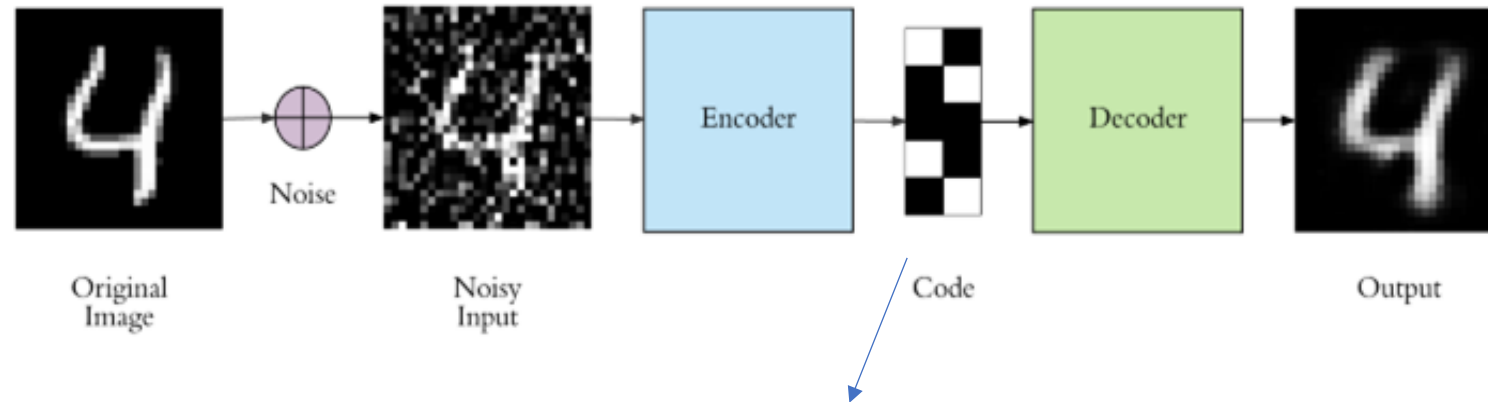
One key ingredient to this success appears to be the use of an unsupervised training criterion to perform a layer-by-layer initialization: each layer is at first trained to produce a higher level (hidden) representation of the observed patterns, based on the representation it receives as input from the layer below, by optimizing a local unsupervised criterion. Each level produces a representation of the input pattern that is more abstract than the previous level’s, because it is obtained by composing more operations. This initialization yields a starting point, from which a global fine-tuning of the model’s parameters is then performed using another training criterion appropriate for the task at hand. This technique has been shown empirically to avoid getting stuck in the kind of poor solutions one typically reaches with random initializations. While unsupervised learning of a mapping that produces “good” intermediate representations of the input pattern seems to be key, little is understood regarding what constitutes “good” representations for initializing deep architectures, or what explicit criteria may guide learning such representations. We know of only a few algorithms that seem to work well for this purpose: Restricted Boltzmann Machines (RBMs) trained with contrastive divergence on one hand, and various types of autoencoders on the other.

The present research begins with the question of what

Appearing in *Proceedings of the 25th International Conference on Machine Learning*, Helsinki, Finland, 2008. Copyright 2008 by the author(s)/owner(s).

DENOISING AUTOENCODERS

- The *latent representation* forces the autoencoder to learn an intelligent representation of the data.
- Another way to force the autoencoder to learn useful features is **by adding random noise** to its inputs and making it recover the **original noise-free data**.
- **Denoising:** input [clean image + noise], train to reproduce the [clean image].



Compressed representation: the feature
we want to extract from the image

DENOISING AUTOENCODERS

- In other words,
 - Basic autoencoder trains to minimize the loss between $\mathbf{x}$ and the reconstruction $g(f(\mathbf{x}))$.
 - Denoising autoencoders train to minimize the loss between $\mathbf{x}$ and $g(f(\mathbf{x}+\mathbf{w}))$, where $\mathbf{w}$ is **random noise**.
- **Same possible architectures**, but different training data.



AUTOENCODERS: APPLICATIONS

Image colorization: input black and white and train to produce color images



AUTOENCODERS: APPLICATIONS

Watermark removal:



AUTOENCODER – APPLICATION



UNIVERSITÀ
di **VERONA**

Anomaly detection:



VARIATIONAL AUTOENCODERS



UNIVERSITÀ
di **VERONA**

arXiv:1312.6114v11 [stat.ML] 10 Dec 2022

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions are two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

1 Introduction

How can we perform efficient approximate inference and learning with directed probabilistic models whose continuous latent variables and/or parameters have intractable posterior distributions? The variational Bayesian (VB) approach involves the optimization of an approximation to the intractable posterior. Unfortunately, the common mean-field approach requires analytical solutions of expectations w.r.t. the approximate posterior, which are also intractable in the general case. We show how a reparameterization of the variational lower bound yields a simple differentiable unbiased estimator of the lower bound; this SGVB (Stochastic Gradient Variational Bayes) estimator can be used for efficient approximate posterior inference in almost any model with continuous latent variables and/or parameters, and is straightforward to optimize using standard stochastic gradient ascent techniques.

For the case of an i.i.d. dataset and continuous latent variables per datapoint, we propose the Auto-Encoding VB (AEVB) algorithm. In the AEVB algorithm we make inference and learning especially efficient by using the SGVB estimator to optimize a recognition model that allows us to perform very efficient approximate posterior inference using simple ancestral sampling, which in turn allows us to efficiently learn the model parameters, without the need of expensive iterative inference schemes (such as MCMC) per datapoint. The learned approximate posterior inference model can also be used for a host of tasks such as recognition, denoising, representation and visualization purposes. When a neural network is used for the recognition model, we arrive at the *variational auto-encoder*.

2 Method

The strategy in this section can be used to derive a lower bound estimator (a stochastic objective function) for a variety of directed graphical models with continuous latent variables. We will restrict ourselves here to the common case where we have an i.i.d. dataset with latent variables per datapoint, and where we like to perform maximum likelihood (ML) or maximum a posteriori (MAP) inference on the (global) parameters, and variational inference on the latent variables. It is, for example,

VARIATIONAL AUTOENCODERS (VAE)

- Instead of encoding an input into a point in the latent space, a VAE encodes it as a distribution over the latent space, i.e.,

Simple AE:

Input $x \rightarrow \text{encoding} \rightarrow$ latent representation $z = e(x) \rightarrow \text{decoding} \rightarrow$ reconstruction of input $d(z)$

VAE:

Input $x \rightarrow \text{encoding} \rightarrow$ latent distribution $p(z|x) \rightarrow \text{sampling} \rightarrow$ sampled latent representation $z \sim p(z|x) \rightarrow \text{decoding} \rightarrow$ reconstruction of input $d(z)$

The distributions p are chosen to be **normal**: the encoder can learn the **mean** and the **covariance matrix**



VARIATIONAL AUTOENCODERS (VAE)

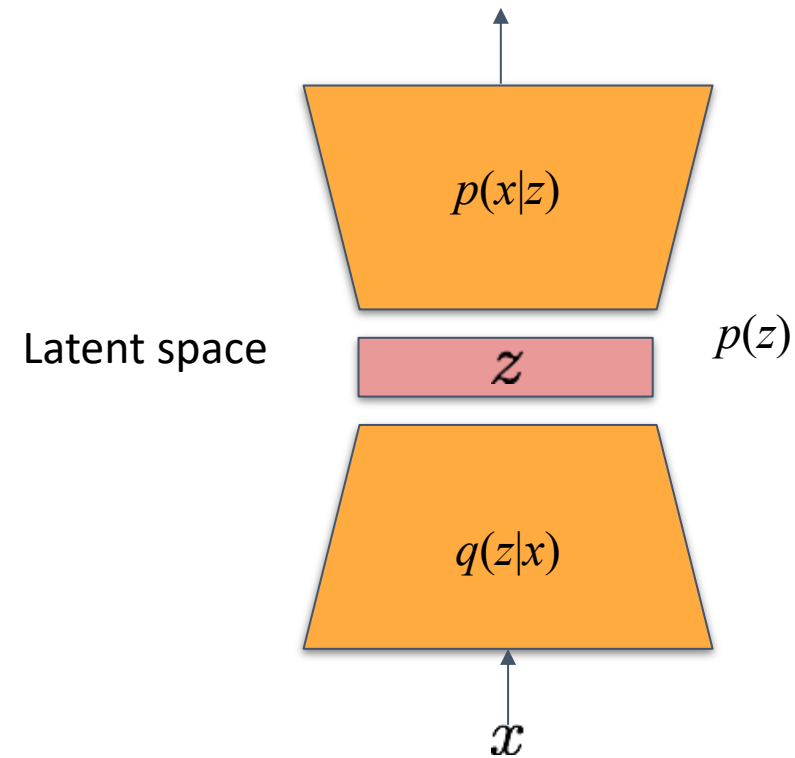
- VAEs define intractable (i.e., either impossible or computationally very expensive) density function with latent z :

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

- Cannot optimize directly, derive and optimize the lower bound on **likelihood** instead.

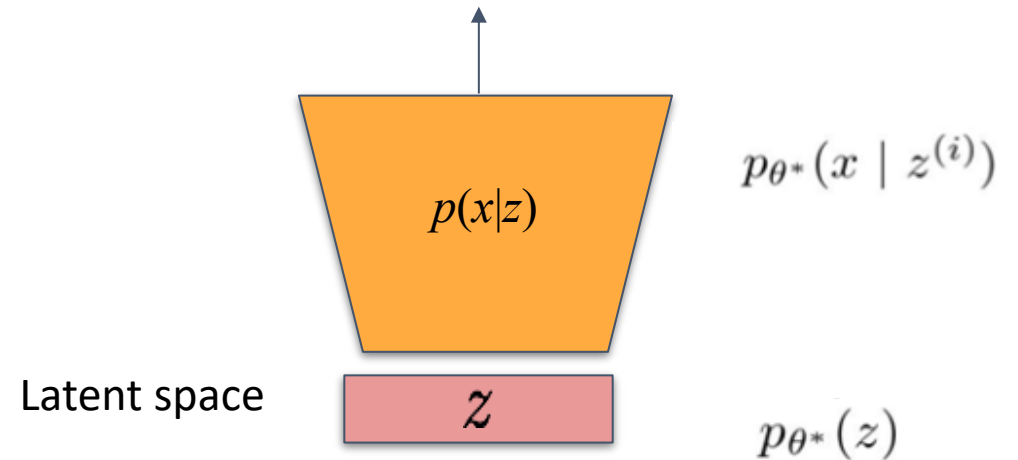
VARIATIONAL AUTOENCODERS (VAE)

- Probabilistic spin on autoencoders: let us sample from the model to generate data!
- In detail, VAE introduces a probabilistic approach to autoencoders by learning a **distribution over the latent space**, allowing us to sample from this distribution to generate new data.



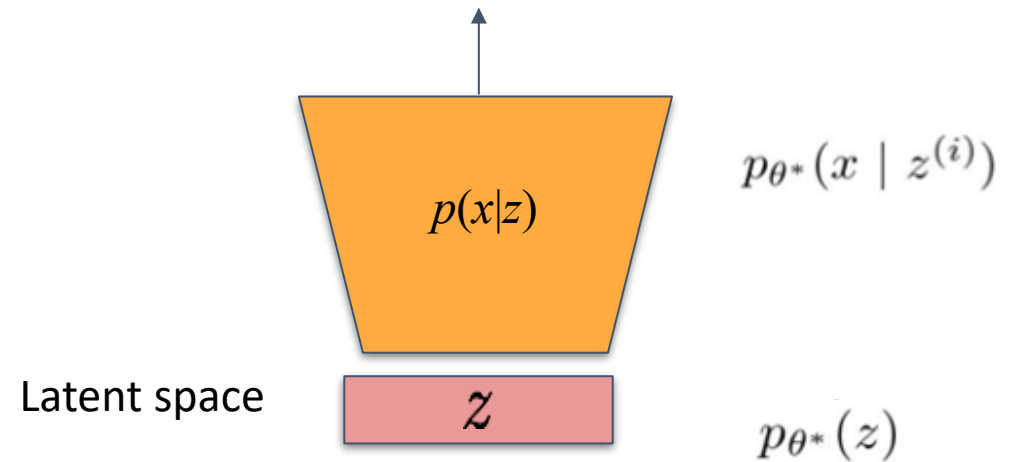
VARIATIONAL AUTOENCODERS (VAE)

- We assume training data representation is generated from underlying **unobserved (latent) z** .
- We want to estimate the optimal parameters of this generative model, according to the **maximum likelihood**.



VARIATIONAL AUTOENCODERS (VAE)

- We can choose **prior** $p(\mathbf{z})$ to be simple, e.g. Gaussian.
- **Conditional** $p(\mathbf{x}|\mathbf{z})$ is complex, meaning it's difficult to model directly with simple functions, so we use a **neural network** to learn how the data (e.g., images) is generated from the latent variables.



VAE: TRAINING

- Directly computing the **likelihood** $p(\mathbf{x})$ is intractable because it requires summing (or integrating) over all possible latent variable configurations. For complex models with many latent variables (e.g., deep generative models), this becomes computationally infeasible.
- Similarly, computing the **posterior** $p(\mathbf{z}|\mathbf{x})$ is intractable because it involves the **marginal likelihood** $p(\mathbf{x})$, which is again a difficult integral to compute.

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})d\mathbf{z}$$

Intractable
to compute
 $p(\mathbf{x}|\mathbf{z})$ for
every $\mathbf{z}$!

Simple
Gaussian
prior

Decoder
neural
network

$$p_{\theta}(\mathbf{z}|\mathbf{x}) = p_{\theta}(\mathbf{x}|\mathbf{z})p_{\theta}(\mathbf{z})/p_{\theta}(\mathbf{x})$$

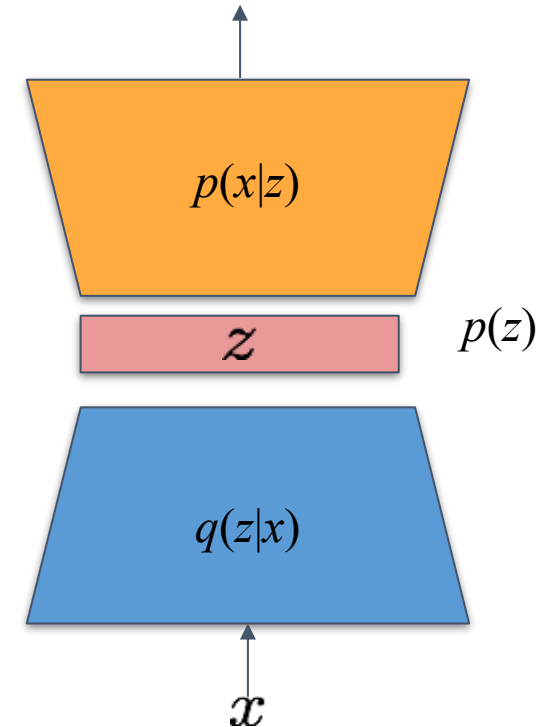
Decoder
neural
network

Simple
gaussian
prior

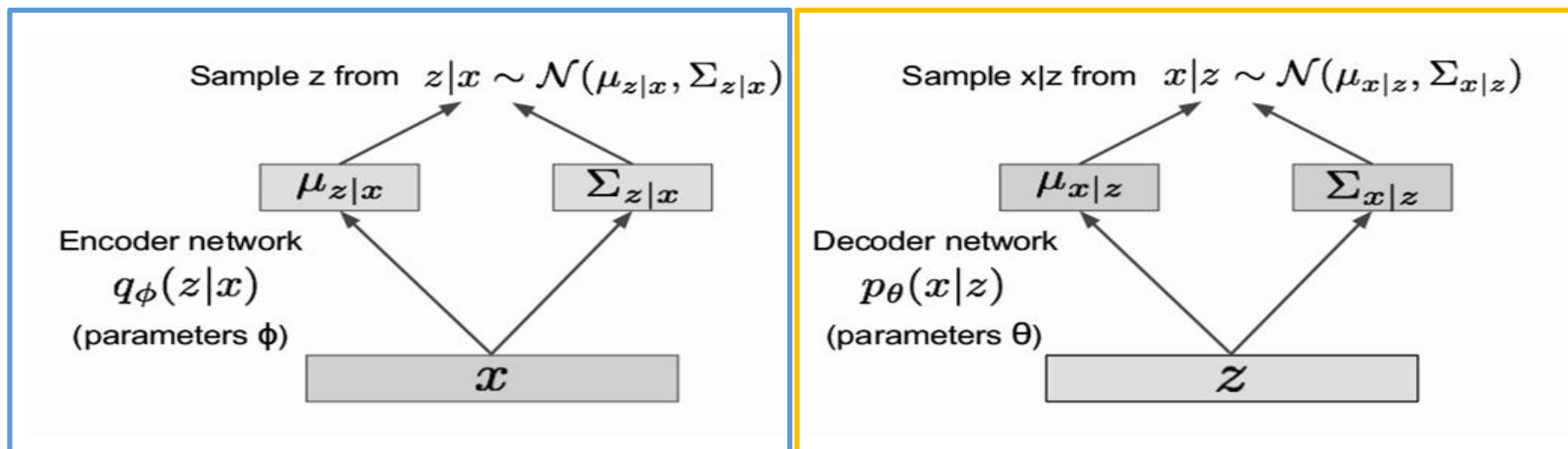
Intractable data
likelihood

VAE: TRAINING

- **Solution:** In addition to decoder network modelling, define an additional **encoder** network that approximates $p_{\theta}(z|x)$
- This allows us to derive a lower bound (ELBO) on the data likelihood that is tractable, and we can optimise.
- Since we are modelling the probabilistic generation of data, the **encoder** and **decoder networks are probabilistic**.



VAE: TRAINING



- **Encoder:** Maps data $\mathbf{x}$ to a distribution $\mathbf{q}_{\phi}(\mathbf{z}|\mathbf{x})$ over latent variables $\mathbf{z}$.
- **Decoder:** Maps latent variables $\mathbf{z}$ to data $\mathbf{x}$, generating data from the distribution $\mathbf{p}_{\theta}(\mathbf{x}|\mathbf{z})$.
- The objective is to maximize: the **Evidence Lower Bound (ELBO)**

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - \text{KL}(q_{\phi}(z | x) || p(z))$$

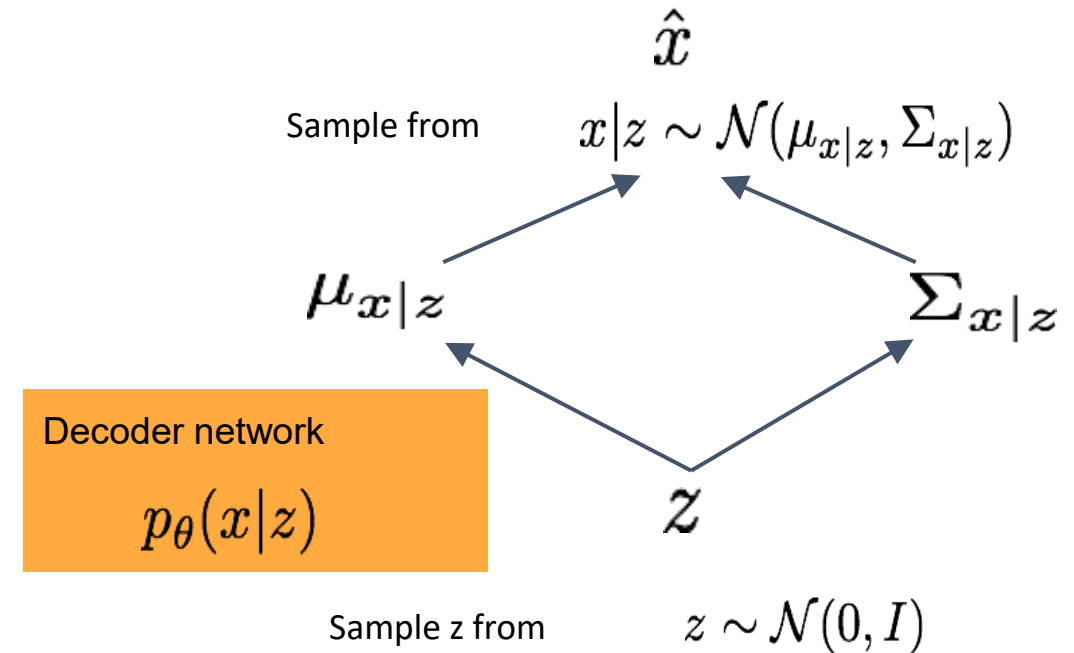
VAE: TRAINING

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)]}_{\text{reconstruction error (decoder)}} - \underbrace{\text{KL}(q_{\phi}(z | x) \| p(z))}_{\text{regularizer}}$$

- **Regularizer**: KL divergence between the approximate posterior $q_{\phi}(z|x)$ and the prior $p(z)$. Aims to make an approximate posterior close to the prior.

VAE: GENERATING DATA

- Sample $\mathbf{z}$ from the prior distribution $\mathbf{p}(\mathbf{z})$, i.e., gaussian
- Use the trained decoder network $\mathbf{p}_{\theta}(\mathbf{x}|\mathbf{z})$ to generate data $\hat{\mathbf{x}}$



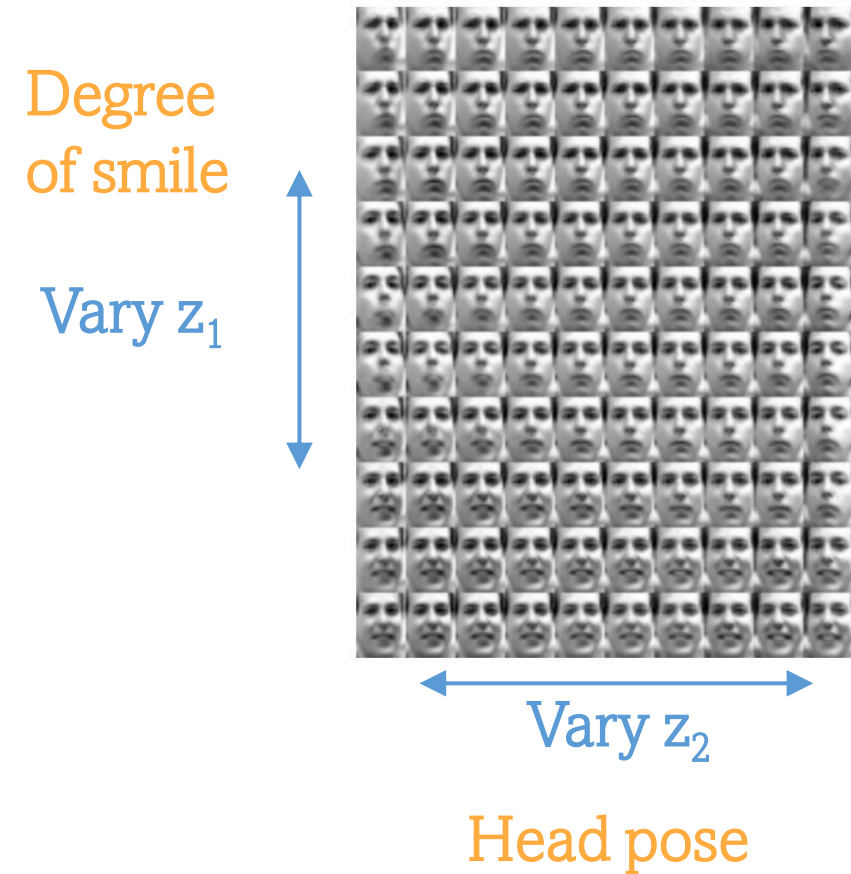


VAE: LATENT SPACE

- The **prior distribution** on the latent variables $\mathbf{z}$ is typically chosen to be **diagonal Gaussian**.
 - This means that each dimension of $\mathbf{z}$ is **independent** of the others. Each latent variable dimension does not depend on the others, and each has its own independent distribution.
 - This is a simplification that makes the optimization of the model easier and makes the model more tractable.

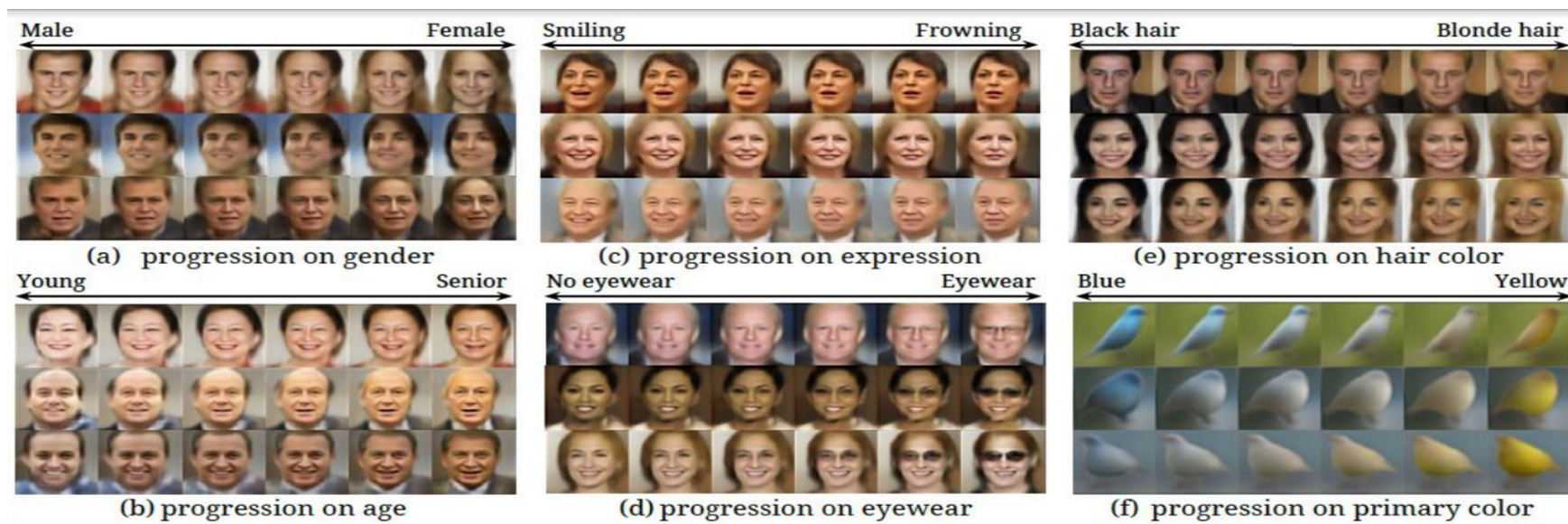
VAE: LATENT SPACE

- Different dimensions of $\mathbf{z}$ encode interpretable factors of variation.
 - I.e., the model learns to encode different factors of variation into different dimensions of the latent space, making them interpretable.



CONDITIONAL VAE

- What if we have labels? (e.g. digit labels or attributes) or other inputs we wish to condition on?
- In a CVAE, the latent variable $\mathbf{z}$ is conditioned on some additional information $\mathbf{y}$ (e.g., labels, attributes, or any extra context).
- None of the derivation changes. Replace $p(x|z)$ with $p(x|z,y)$ and $q(x|z)$ with $q(x|z,y)$.





VAE SUMMARY

- Probabilistic spin to traditional autoencoders: allows generating data.
- Defines an intractable density $\Rightarrow$ derive and optimise a (variational) lower bound.
- **Pros:**
 - Principled approach to generative models.
 - Allows inference of $q(z|x)$, which can be a useful feature representation for other tasks.
- **Cons:**
 - Maximises the lower bound of likelihood.
 - Originally, samples were blurrier and of lower quality compared to GANs (next model).
- Relatively recent areas of research:
 - Novel models (e.g., VQ-VAE) and combination with new architectures (e.g., with transformer).

GENERATIVE ADVERSARIAL NETWORKS (GANS)



UNIVERSITÀ
di **VERONA**

arXiv:1406.2661v1 [stat.ML] 10 Jun 2014

Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair, Aaron Courville, Yoshua Bengio¹
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

1 Introduction

The promise of deep learning is to discover rich, hierarchical models [2] that represent probability distributions over the kinds of data encountered in artificial intelligence applications, such as natural images, audio waveforms containing speech, and symbols in natural language corpora. So far, the most striking successes in deep learning have involved discriminative models, usually those that map a high-dimensional, rich sensory input to a class label [14, 22]. These striking successes have primarily been based on the backpropagation and dropout algorithms, using piecewise linear units [19, 9, 10] which have a particularly well-behaved gradient. Deep *generative* models have had less of an impact, due to the difficulty of approximating many intractable probabilistic computations that arise in maximum likelihood estimation and related strategies, and due to difficulty of leveraging the benefits of piecewise linear units in the generative context. We propose a new generative model estimation procedure that sidesteps these difficulties.¹

In the proposed *adversarial nets* framework, the generative model is pitted against an adversary: a discriminative model that learns to determine whether a sample is from the model distribution or the data distribution. The generative model can be thought of as analogous to a team of counterfeiters, trying to produce fake currency and use it without detection, while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles.

¹Jean Pouget-Abadie is visiting Université de Montréal from Ecole Polytechnique.

²Sherjil Ozair is visiting Université de Montréal from Indian Institute of Technology Delhi

³Yoshua Bengio is a CIFAR Senior Fellow.

⁴All code and hyperparameters available at <http://www.github.com/goodfeli/adversarial>

GENERATIVE ADVERSARIAL NETWORKS (GANs)

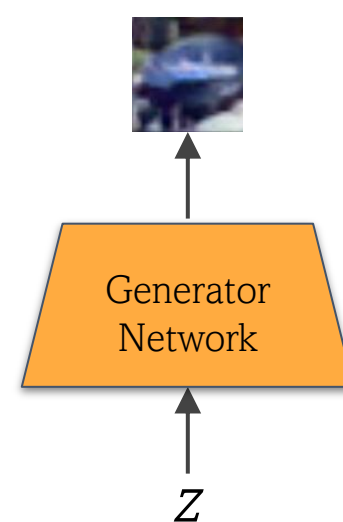


UNIVERSITÀ
di **VERONA**

- GANs **do not** consider any explicit density function.
- GANs just focus on the ability of the model to sample.
 - How to sample from a **complex high-dimensional** training distribution?

GAN: IDEA

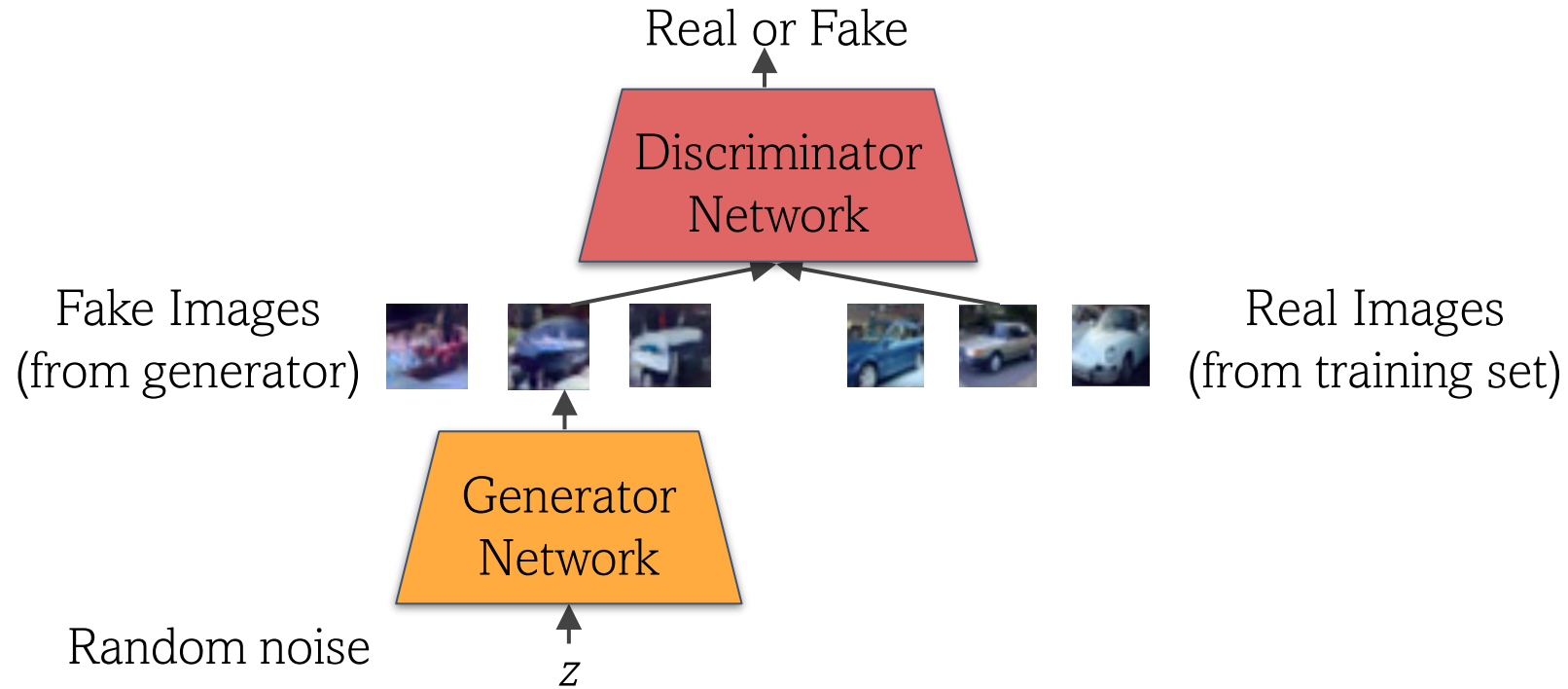
- Sample from a simple distribution, e.g. random noise.
- Train a neural network (***the generator***) to transform that noise into samples from your desired distribution (e.g., human faces, handwritten digits, etc.).



Output: Sample from training distribution

Input: Random noise

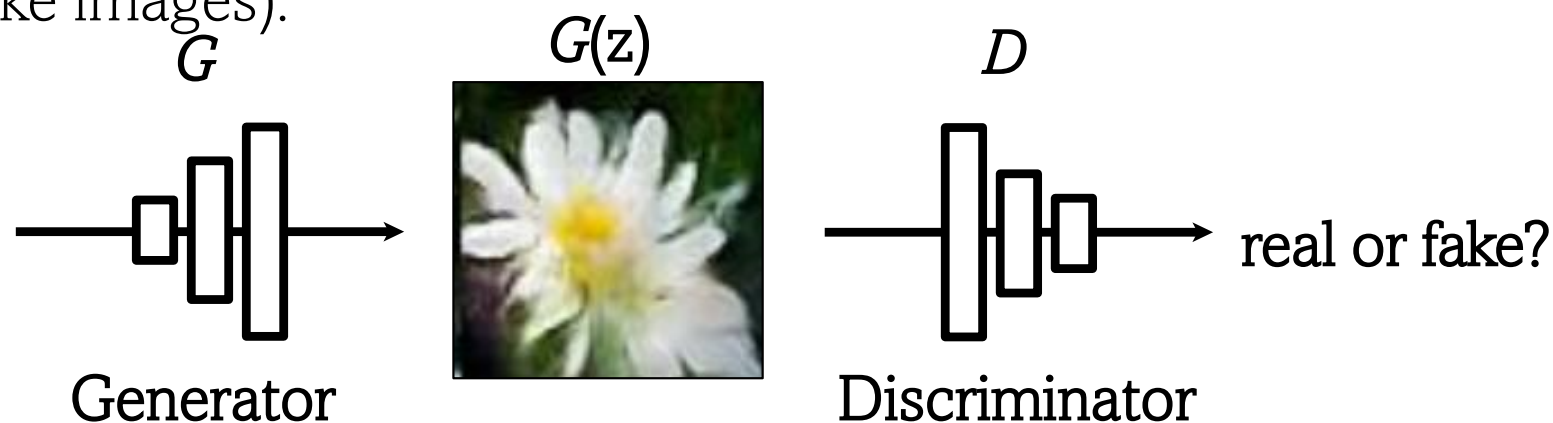
GAN: TWO PLAYERS GAME



- Two neural networks:
 - A **Generator** (G) that learns the mapping from noise to data.
 - A **Discriminator** (D) that tries to tell real data apart from fake data generated by G.

GAN: TWO PLAYERS GAME

- **Discriminator:** Tries to distinguish between real and fake images.
- **Generator:** Tries to fool the discriminator by generating realistic images (so it synthesizes fake images).



- By learning the distribution of the training set, GAN can generate new data never seen before.
- GAN is based on a game theoretic scenario in which a **generator network** must **compete** against an **adversary**.

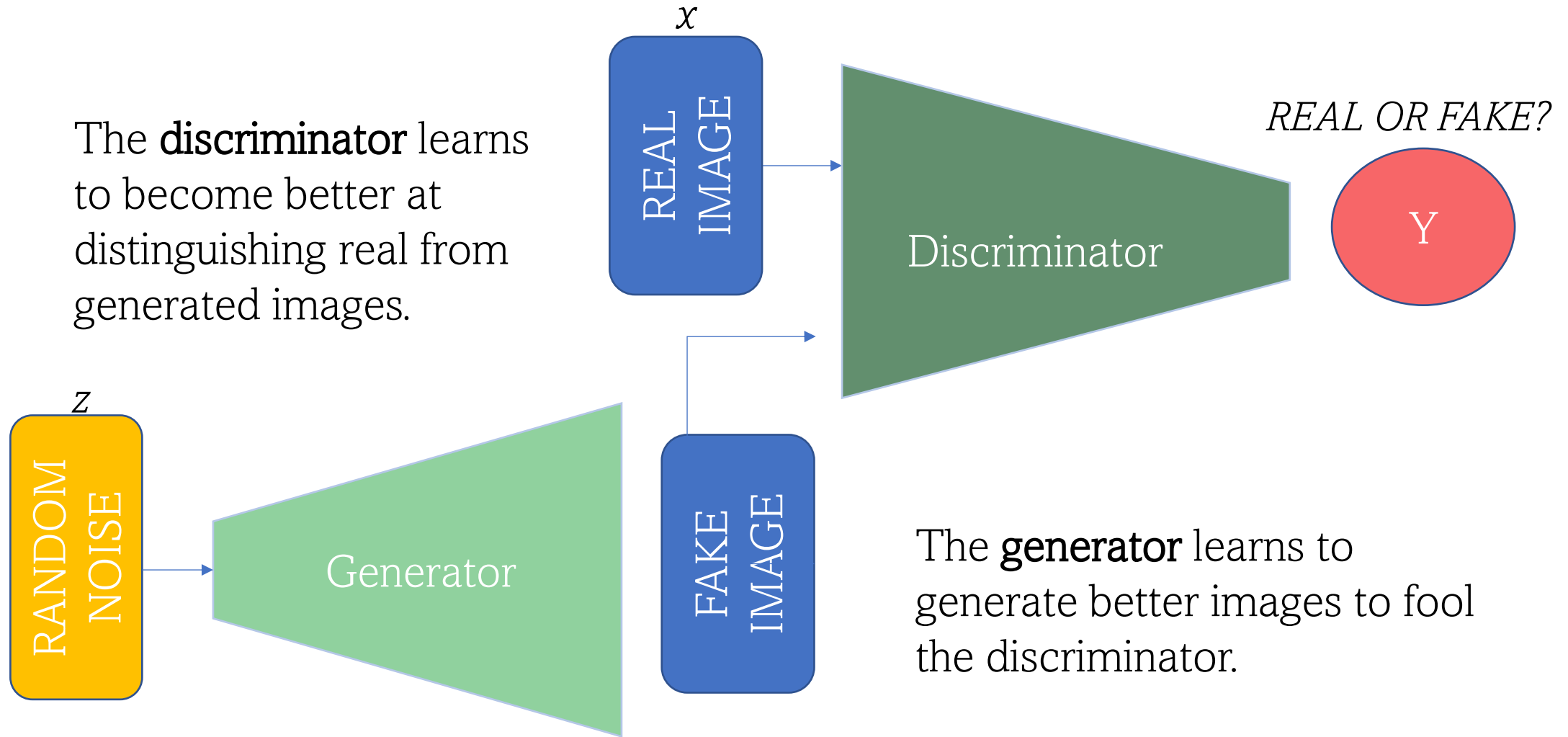


GAN: TWO PLAYERS GAME

- In detail,
 - A **generator network** directly produces samples.
 - Its adversary, the **discriminator network**, attempts to distinguish between samples drawn from the training data and samples drawn from the generator.
 - The **discriminator** estimates a probability value evaluating how likely is that the sample is a real training example rather than a fake sample drawn from the model.

GAN: SUMMARY

The **discriminator** learns to become better at distinguishing real from generated images.



The **generator** learns to generate better images to fool the discriminator.

GAN: OBJECTIVE FUNCTION

- The generator and discriminator are trained **jointly**.
- The objective function is **minmax**.

$$\min_{\theta_g} \max_{\theta_d} \left[\underbrace{\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x)}_{\text{Discriminator output for real data}} + \underbrace{\mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))}_{\text{Discriminator output for generated fake data } G(z)} \right]$$

Discriminator output
for real data

Discriminator output for
generated fake data $G(z)$

- The **Discriminator** outputs likelihood in $(0,1)$ of the real image.
- The **Discriminator** wants $D(x)$ to be close to 1 and $D(G(z))$ is close to 0 $\rightarrow$ tries to maximize the objective function.
- The **Generator** wants $D(G(z))$ is close to 1 $\rightarrow$ tries to minimize the objective function.
- Minmax is solved through alternating gradient descent: the parameters θ_d and θ_g are updated iteratively.

GAN: TRAINING

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

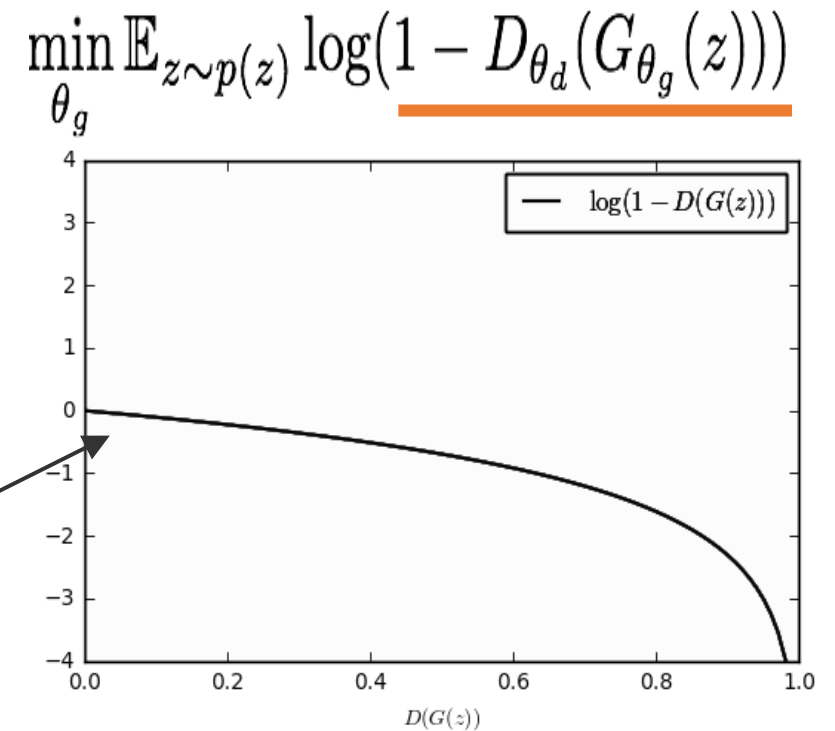
2. **Gradient descent** on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

GAN: TRAINING

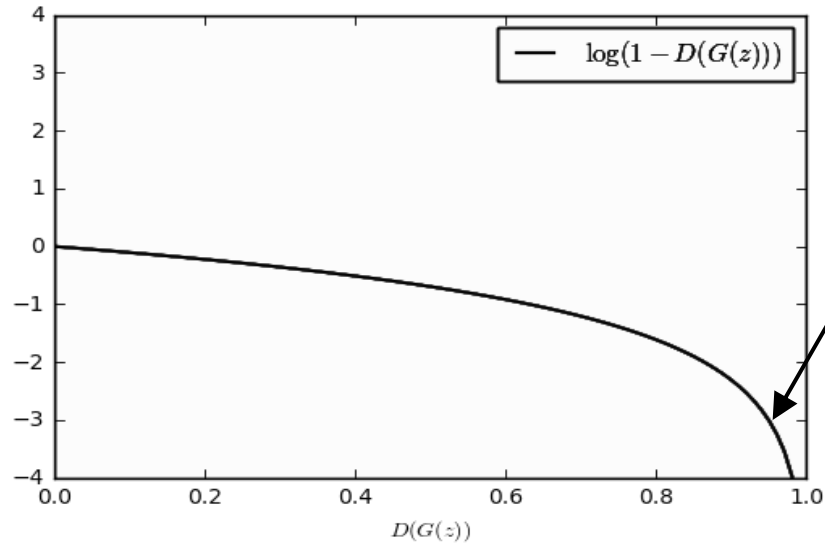
- When the **generator** produces a sample that is clearly fake.
- The **discriminator** sees it and confidently says: "this is fake" (e.g., outputs a value near 0).
- Now, the generator needs to use this signal to **improve**, i.e., backpropagate the gradient to adjust its weights.
- But: when the discriminator is **very confident** the sample is fake, the **gradient it provides is very small (flat)**.
- This means the generator doesn't get a **strong learning signal**, even though the sample is bad and should improve a lot.

This is part of what's called the **vanishing gradient problem** in GAN training.



GAN: TRAINING

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$



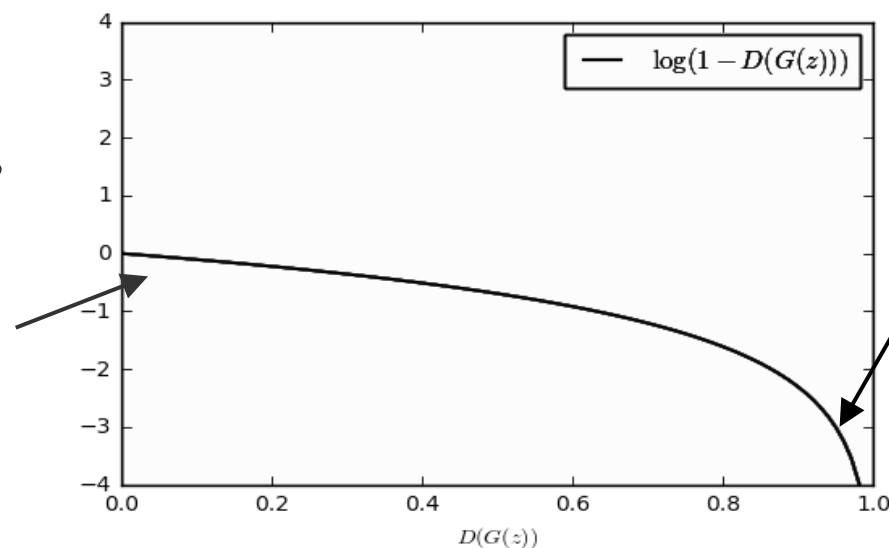
- When a generated sample is already close to looking real, the discriminator is less confident.
- In this uncertain region, the **gradient** is **larger**.
- So ironically, the generator learns more from already good samples than from bad ones.

GAN: TRAINING



$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

When sample is likely fake, want to learn from it to improve generator. But gradient in this region is relatively flat!

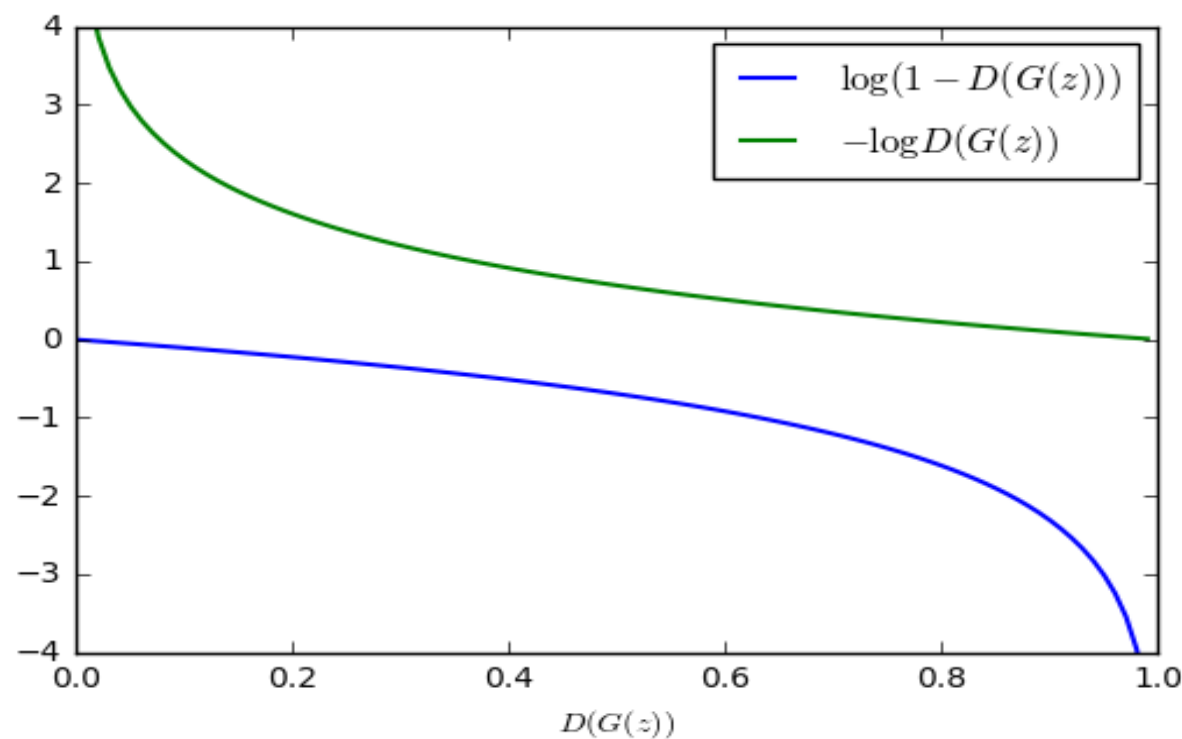


Gradient signal dominated by the region where the sample is already good

GAN: TRAINING

- Instead of minimizing the likelihood of the discriminator being correct, now **maximize** the likelihood of the discriminator being wrong.
- Gradient ascent on generator (Step 2)

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$



GAN: TRAINING

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Alternate between:

1. **Gradient ascent** on discriminator

$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

2. **Gradient ascent** on generator

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$



ISSUES WITH GAN TRAINING

- Hard to train
 - Early in training, when most generated samples are bad, learning is slow because gradients are tiny.
 - Later, as generated samples improve, learning becomes more effective — but only around samples that are already close to real.
 - This imbalance can make GANs **unstable** or **slow** to train.
- Parameters can oscillate or diverge, generator loss does not correlate with sample quality.
- Behavior very sensitive to hyper-parameter selection.
- Several improvements proposed over the years, mostly changing the loss function.

GANs

Nearest real image
for sample to the
left



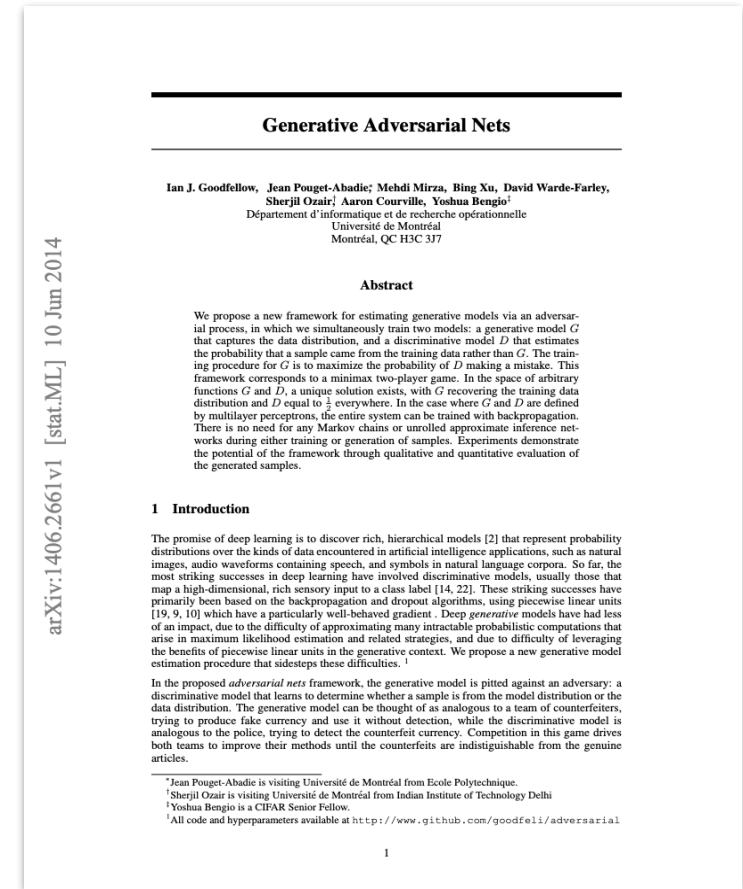
MNIST digits



Toronto Face
Dataset

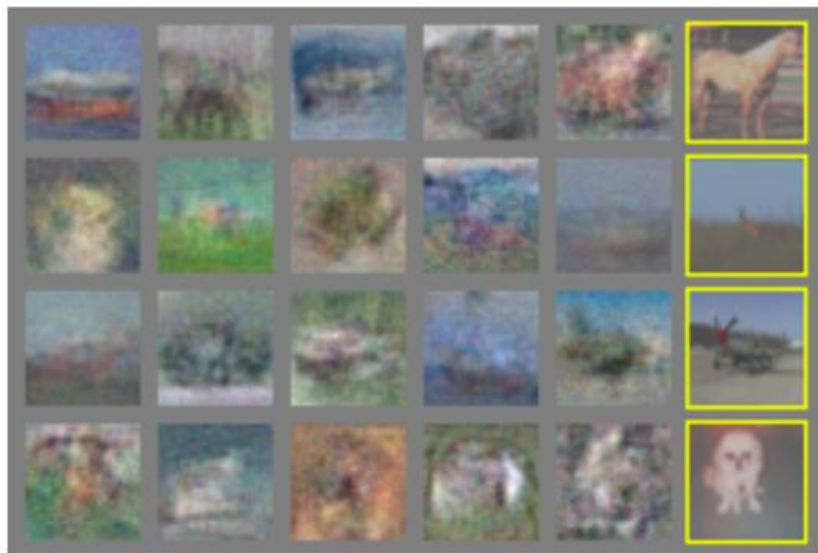


UNIVERSITÀ
di **VERONA**



GANs

Nearest
real image
for sample
to the left



CIFAR-10 (FC networks)



CIFAR-10 (conv networks)



UNIVERSITÀ
di **VERONA**

Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair, Aaron Courville, Yoshua Bengio^{*}
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

1 Introduction

The promise of deep learning is to discover rich, hierarchical models [2] that represent probability distributions over the kinds of data encountered in artificial intelligence applications, such as natural images, audio waveforms containing speech, and symbols in natural language corpora. So far, the most striking successes in deep learning have involved discriminative models, usually those that map a high-dimensional, rich sensory input to a class label [14, 22]. These striking successes have primarily been based on the backpropagation and dropout algorithms, using piecewise linear units [19, 9, 10] which have a particularly well-behaved gradient. Deep generative models have had less of an impact, due to the difficulty of approximating many intractable probabilistic computations that arise in maximum likelihood estimation and related strategies, and due to difficulty of leveraging the benefits of piecewise linear units in the generative context. We propose a new generative model estimation procedure that sidesteps these difficulties.¹

In the proposed *adversarial nets* framework, the generative model is pitted against an adversary: a discriminative model that learns to determine whether a sample is from the model distribution or the data distribution. The generative model can be thought of as analogous to a team of counterfeiters, trying to produce fake currency and use it without detection, while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles.

^{*}Jean Pouget-Abadie is visiting Université de Montréal from Ecole Polytechnique.

[†]Sherjil Ozair is visiting Université de Montréal from Indian Institute of Technology Delhi

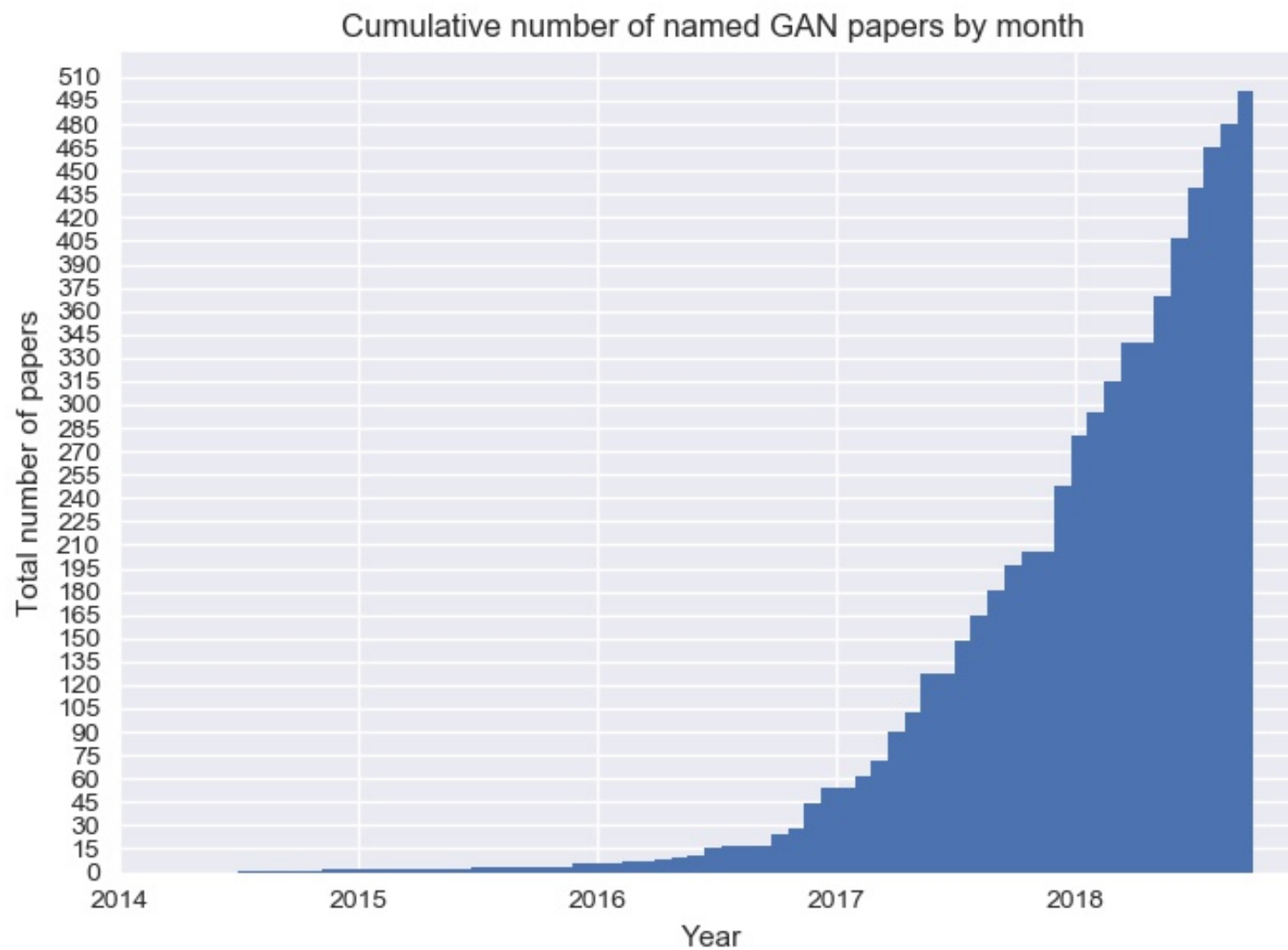
[‡]Yoshua Bengio is a CIFAR Senior Fellow.

[§]All code and hyperparameters available at <http://www.github.com/goodfeli/adversarial>

GAN Zoo



UNIVERSITÀ
di **VERONA**



GAN Zoo

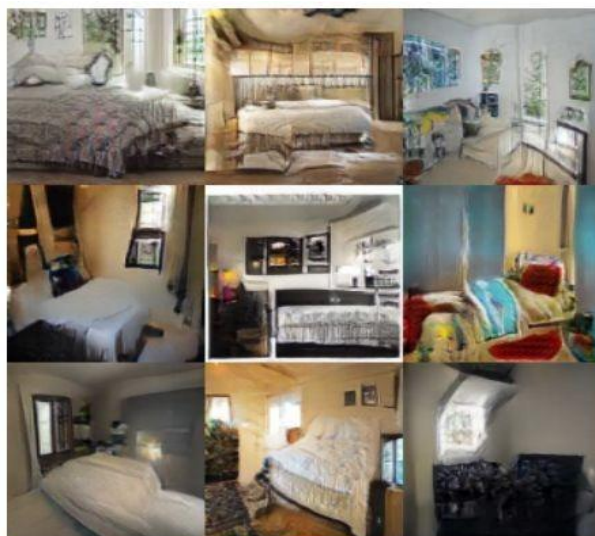


UNIVERSITÀ
di **VERONA**

- GAN - Generative Adversarial Networks
- 3D-GAN - Learning a Probabilistic Latent Space of Object Shapes via 3D Generative-Adversarial Modeling
- acGAN - Face Aging With Conditional Generative Adversarial Networks
- AC-GAN - Conditional Image Synthesis With Auxiliary Classifier GANs
- AdaGAN - AdaGAN: Boosting Generative Models
- AEGAN - Learning Inverse Mapping by Autoencoder based Generative Adversarial Nets
- AffGAN - Amortised MAP Inference for Image Super-resolution
- AL-CGAN - Learning to Generate Images of Outdoor Scenes from Attributes and Semantic Layouts
- ALI - Adversarially Learned Inference
- AM-GAN - Generative Adversarial Nets with Labeled Data by Activation Maximization
- AnoGAN - Unsupervised Anomaly Detection with Generative Adversarial Networks to Guide Marker Discovery
- ArtGAN - ArtGAN: Artwork Synthesis with Conditional Categorical GANs
- b-GAN - b-GAN: Unified Framework of Generative Adversarial Networks
- Bayesian GAN - Deep and Hierarchical Implicit Models
- BEGAN - BEGAN: Boundary Equilibrium Generative Adversarial Networks
- BiGAN - Adversarial Feature Learning
- BS-GAN - Boundary-Seeking Generative Adversarial Networks
- CGAN - Conditional Generative Adversarial Nets
- CaloGAN - CaloGAN: Simulating 3D High Energy Particle Showers in Multi-Layer Electromagnetic Calorimeters with Generative Adversarial Networks
- CCGAN - Semi-Supervised Learning with Context-Conditional Generative Adversarial Networks
- CatGAN - Unsupervised and Semi-supervised Learning with Categorical Generative Adversarial Networks
- CoGAN - Coupled Generative Adversarial Networks
- Context-RNN-GAN - Contextual RNN-GANs for Abstract Reasoning Diagram Generation
- C-RNN-GAN - C-RNN-GAN: Continuous recurrent neural networks with adversarial training
- CS-GAN - Improving Neural Machine Translation with Conditional Sequence Generative Adversarial Nets
- CVAE-GAN - CVAE-GAN: Fine-Grained Image Generation through Asymmetric Training
- CycleGAN - Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks
- DTN - Unsupervised Cross-Domain Image Generation
- DCGAN - Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks
- DiscoGAN - Learning to Discover Cross-Domain Relations with Generative Adversarial Networks
- DR-GAN - Disentangled Representation Learning GAN for Pose-Invariant Face Recognition
- DualGAN - DualGAN: Unsupervised Dual Learning for Image-to-Image Translation
- EBGAN - Energy-based Generative Adversarial Network
- f-GAN - f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization
- FF-GAN - Towards Large-Pose Face Frontalization in the Wild
- GAWWN - Learning What and Where to Draw
- GeneGAN - GeneGAN: Learning Object Transfiguration and Attribute Subspace from Unpaired Data
- Geometric GAN - Geometric GAN
- GoGAN - Gang of GANs: Generative Adversarial Networks with Maximum Margin Ranking
- GP-GAN - GP-GAN: Towards Realistic High-Resolution Image Blending
- IAN - Neural Photo Editing with Introspective Adversarial Networks
- iGAN - Generative Visual Manipulation on the Natural Image Manifold
- IcGAN - Invertible Conditional GANs for image editing
- ID-CGAN - Image De-raining Using a Conditional Generative Adversarial Network
- Improved GAN - Improved Techniques for Training GANs
- InfoGAN - InfoGAN: Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets
- LAGAN - Learning Particle Physics by Example: Location-Aware Generative Adversarial Networks for Physics Synthesis
- LAPGAN - Deep Generative Image Models using a Laplacian Pyramid of Adversarial Networks

2017: EXPLOSION OF GANS

Better training and generation



LSGAN, Zhu 2017.



Wasserstein GAN,
Arjovsky 2017.
Improved Wasserstein
GAN, Gulrajani 2017.



Progressive GAN, Karras 2018.



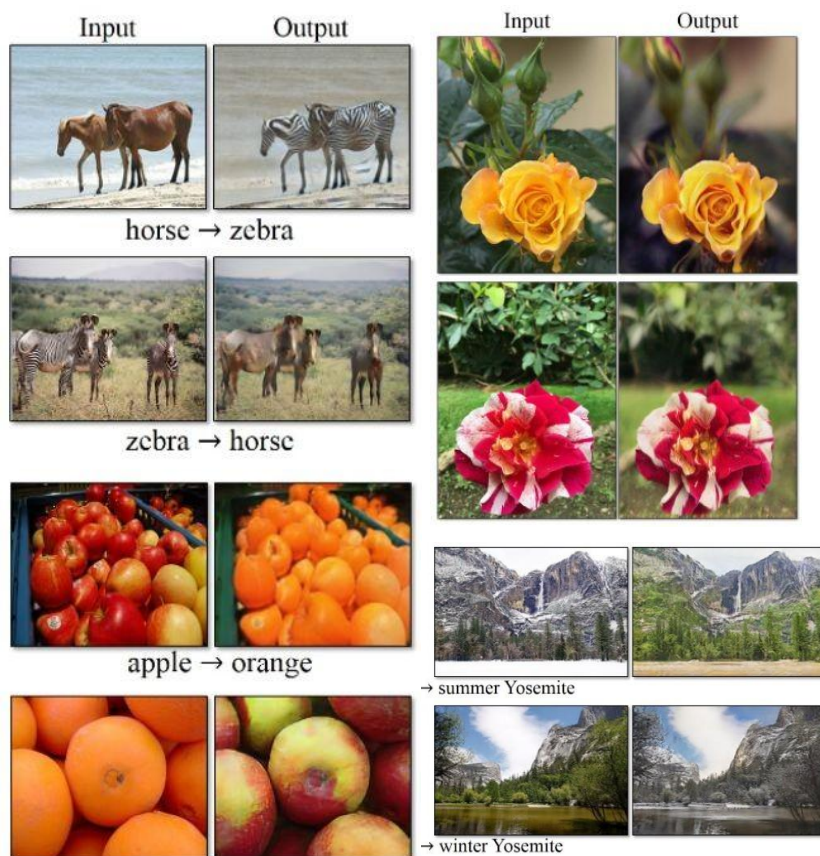
UNIVERSITÀ
di **VERONA**

2017: EXPLOSION OF GANS



UNIVERSITÀ
di **VERONA**

Source → Target domain transfer



CycleGAN. Zhu et al. 2017

Text → Image Synthesis

this small bird has a pink
breast and crown, and black
primaries and secondaries.

this magnificent fellow is
almost all black with a red
crest, and white cheek patch.



Reed et al. 2017

Many GAN applications



Pix2pix. Isola 2017.

Many examples at <https://phillipi.github.io/pix2pix/>

2019: BIGGAN



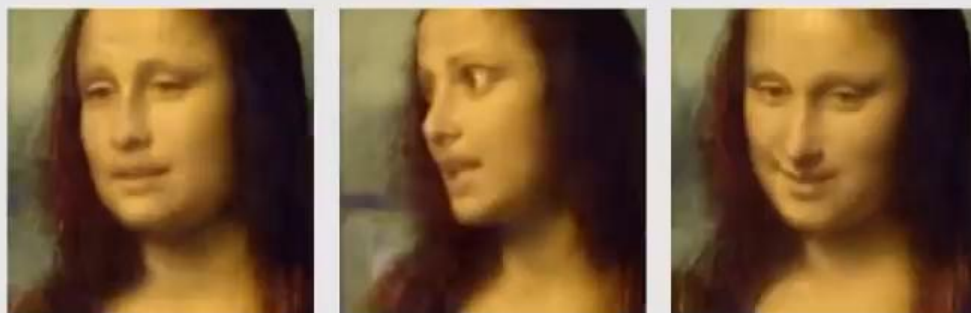
UNIVERSITÀ
di **VERONA**

Brock et al. "Large Scale GAN Training for High Fidelity Natural Image Synthesis" (2019)



<https://arxiv.org/pdf/1809.11096.pdf>

Living portraits



“Unsupervised Image to Image Translation Networks” (2018)



<https://www.youtube.com/watch?v=9VC0c3pndbI>

https://www.youtube.com/watch?v=Z_Rxf0TfBJE

DCGAN



UNIVERSITÀ
di **VERONA**

arXiv:1511.06434v2 [cs.LG] 7 Jan 2016

Under review as a conference paper at ICLR 2016

UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS

Alec Radford & Luke Metz
Indico Research
Boston, MA
{alec, luke}@indico.io

Soumith Chintala
Facebook AI Research
New York, NY
soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning. Training on various image datasets, we show convincing evidence that our deep convolutional adversarial pair learns a hierarchy of representations from object parts to scenes in both the generator and discriminator. Additionally, we use the learned features for novel tasks - demonstrating their applicability as general image representations.

1 INTRODUCTION

Learning reusable feature representations from large unlabeled datasets has been an area of active research. In the context of computer vision, one can leverage the practically unlimited amount of unlabeled images and videos to learn good intermediate representations, which can then be used on a variety of supervised learning tasks such as image classification. We propose that one way to build good image representations is by training Generative Adversarial Networks (GANs) (Goodfellow et al., 2014), and later reusing parts of the generator and discriminator networks as feature extractors for supervised tasks. GANs provide an attractive alternative to maximum likelihood techniques. One can additionally argue that their learning process and the lack of a heuristic cost function (such as pixel-wise independent mean-square error) are attractive to representation learning. GANs have been known to be unstable to train, often resulting in generators that produce nonsensical outputs. There has been very limited published research in trying to understand and visualize what GANs learn, and the intermediate representations of multi-layer GANs.

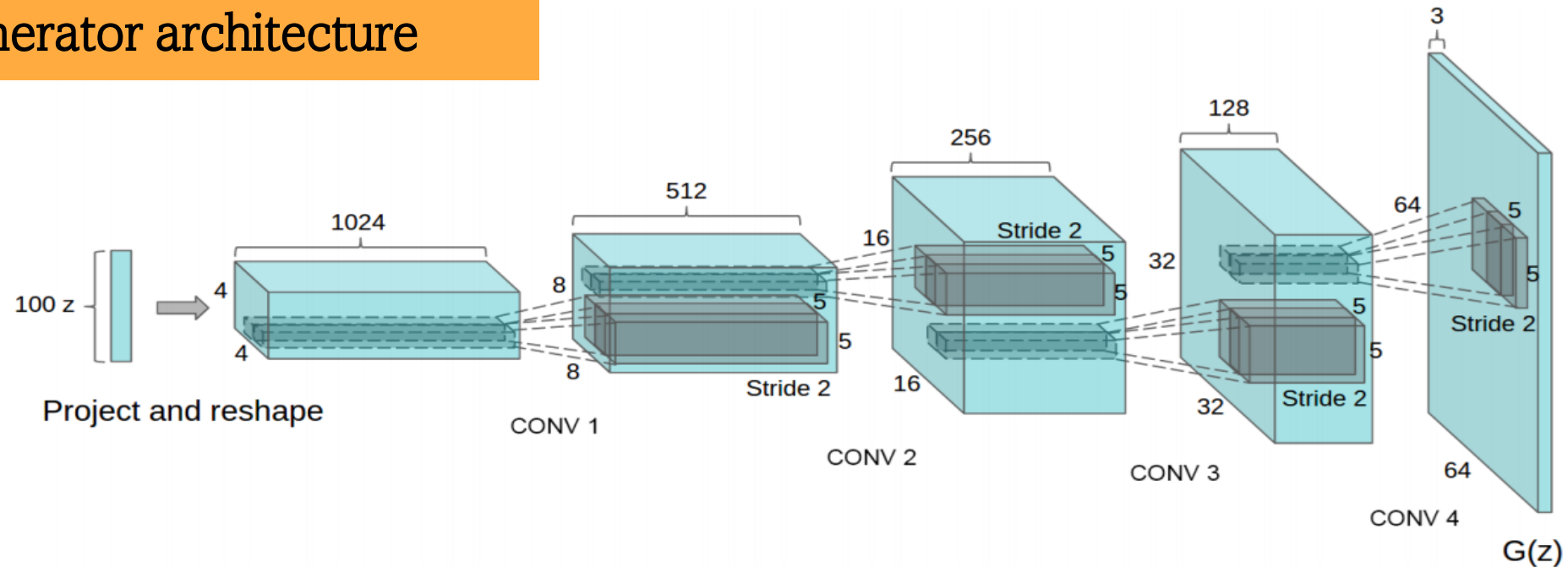
In this paper, we make the following contributions

- We propose and evaluate a set of constraints on the architectural topology of Convolutional GANs that make them stable to train in most settings. We name this class of architectures Deep Convolutional GANs (DCGAN)
- We use the trained discriminators for image classification tasks, showing competitive performance with other unsupervised algorithms.
- We visualize the filters learnt by GANs and empirically show that specific filters have learned to draw specific objects.

DCGAN

- Propose principles for designing **convolutional architectures** for GANs, the first model for large-resolution image generation.

Generator architecture



DCGAN

Discriminator architecture

- No pooling, only strided convolutions
- Use Leaky ReLU activations (sparse gradients cause problems for training)
- Use only one FC layer before the softmax output
- Use batch normalization after most layers (in the generator too)

DCGAN: ALL IN ONE

- The generator is an **upsampling network** with fractionally strided convolutions.
- Discriminator is a **convolutional network**.

Architecture guidelines for stable Deep Convolutional GANs

- Replace any pooling layers with strided convolutions (discriminator) and fractional-strided convolutions (generator).
- Use batchnorm in both the generator and the discriminator.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in generator for all layers except for the output, which uses Tanh.
- Use LeakyReLU activation in the discriminator for all layers.

DCGAN: RESULTS



UNIVERSITÀ
di **VERONA**

Generated bedrooms

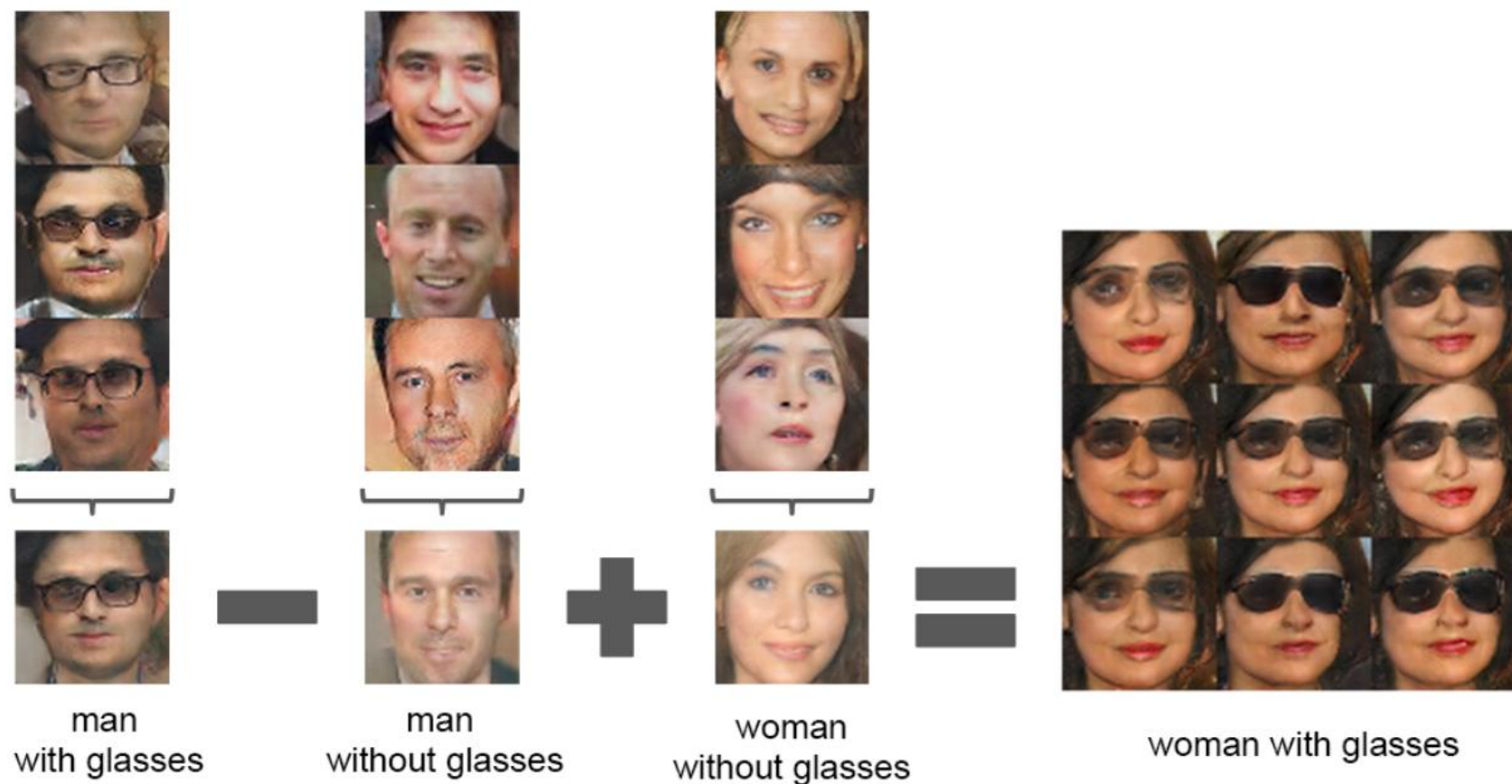


Radford et al, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks", ICLR 2016

ARITHMETIC OF GAN

- The generator takes in a vector $\mathbf{z}$ (random noise from latent space).
- Each point in this latent space maps to a generated sample.
- If the generator learns a smooth and meaningful latent space, then:
- Moving in certain directions in that space changes specific attributes of the sample.
- Example: Let's say you have three latent vectors:
 - $\mathbf{z}_1 \rightarrow$ generates an image of a man
 - $\mathbf{z}_2 \rightarrow$ generates an image of a man with glasses
 - $\mathbf{z}_3 = \mathbf{z}_2 - \mathbf{z}_1 + \mathbf{z}_4$, where $\mathbf{z}_4$ generates a woman
 - Then $G(\mathbf{z}_3)$ often generates **a woman with glasses**.
- **This is the arithmetic**: treating semantic changes as vector directions in latent space.

ARITHMETIC OF GAN (EXAMPLE)



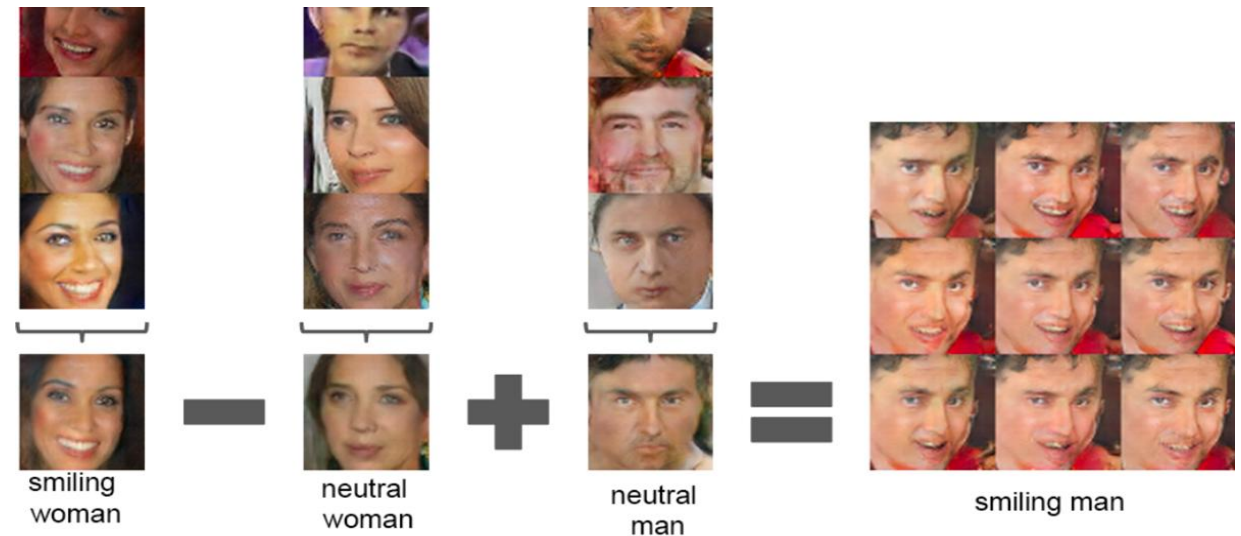


ARITHMETIC OF GAN

- Why does this happen?
 - Because during training, the GAN **implicitly learns high-level concepts** (like gender, glasses, hair) and organizes them as linear directions in the latent space.
- You can edit images by moving in certain directions in latent space (e.g., add smile, change age). This forms the basis for latent space manipulation and GAN-based image editing tools.

ARITHMETIC OF GAN (EXAMPLE)

- For each column, the z vectors of samples are averaged.
- Arithmetic is performed on the mean vectors creating a vector y .
- The center sample on the right hand side is produced by feeding the new vector y as input to the generator.
- Interpolation capabilities of the generator, uniform noise sampled was added to y to produce the 8 other samples.



EVALUATION OF A GAN



UNIVERSITÀ
di **VERONA**

arXiv:1511.01844v3 [stat.ML] 24 Apr 2016

Published as a conference paper at ICLR 2016

A NOTE ON THE EVALUATION OF GENERATIVE MODELS

Lucas Theis*
University of Tübingen
72072 Tübingen, Germany
lucas@bethgelab.org

Aäron van den Oord*[†]
Ghent University
9000 Ghent, Belgium
aaron.vandenoord@ugent.be

Matthias Bethge
University of Tübingen
72072 Tübingen, Germany
matthias@bethgelab.org

ABSTRACT

Probabilistic generative models can be used for compression, denoising, inpainting, texture synthesis, semi-supervised learning, unsupervised feature learning, and other tasks. Given this wide range of applications, it is not surprising that a lot of heterogeneity exists in the way these models are formulated, trained, and evaluated. As a consequence, direct comparison between models is often difficult. This article reviews mostly known but often underappreciated properties relating to the evaluation and interpretation of generative models with a focus on image models. In particular, we show that three of the currently most commonly used criteria—average log-likelihood, Parzen window estimates, and visual fidelity of samples—are largely independent of each other when the data is high-dimensional. Good performance with respect to one criterion therefore need not imply good performance with respect to the other criteria. Our results show that extrapolation from one criterion to another is not warranted and generative models need to be evaluated directly with respect to the application(s) they were intended for. In addition, we provide examples demonstrating that Parzen window estimates should generally be avoided.

1 INTRODUCTION

Generative models have many applications and can be evaluated in many ways. For density estimation and related tasks, log-likelihood (or equivalently Kullback-Leibler divergence) has been the de-facto standard for training and evaluating generative models. However, the likelihood of many interesting models is computationally intractable. For example, the normalization constant of unnormalized energy-based models is generally difficult to compute, and latent-variable models often require us to solve complex integrals to compute the likelihood. These models may still be trained with respect to a different objective that is more or less related to log-likelihood, such as contrastive divergence (Hinton, 2002), score matching (Hyvärinen, 2005), lower bounds on the log-likelihood (Bishop, 2006), noise-contrastive estimation (Gutmann & Hyvärinen, 2010), probability flow (Sohl-Dickstein et al., 2011), maximum mean discrepancy (MMD) (Gretton et al., 2007; Li et al., 2015), or approximations to the Jensen-Shannon divergence (JSD) (Goodfellow et al., 2014).

For computational reasons, generative models are also often compared in terms of properties more readily accessible than likelihood, even when the task is density estimation. Examples include visualizations of model samples, interpretations of model parameters (Hyvärinen et al., 2009), Parzen window estimates of the model’s log-likelihood (Breuleux et al., 2009), and evaluations of model performance in surrogate tasks such as denoising or missing value imputation.

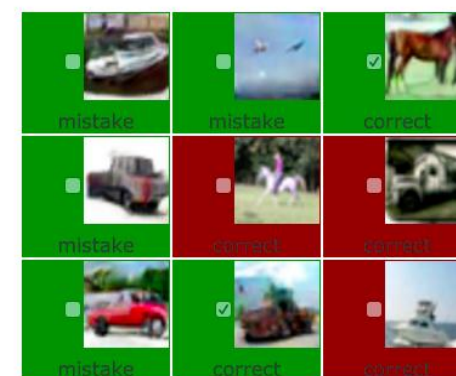
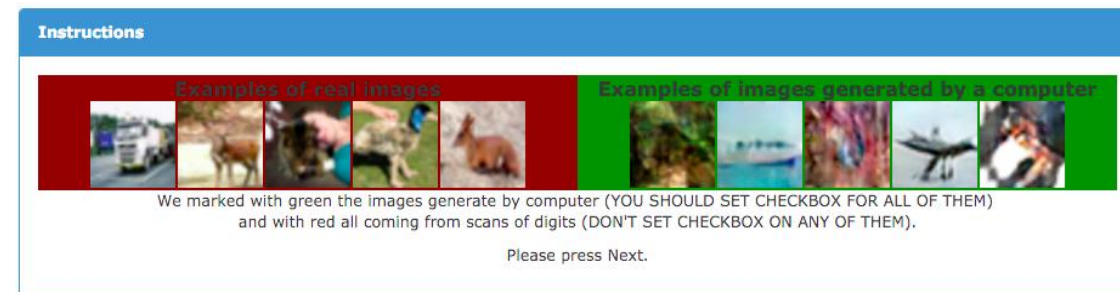
In this paper, we look at some of the implications of choosing certain training and evaluation criteria. We first show that training objectives such as JSD and MMD can result in very different optima than

*These authors contributed equally to this work.

[†]Now at Google DeepMind.

EVALUATION OF A GAN

- Evaluation is HARD!!!
- Showing pictures of samples (**user study**) is not enough, especially for simpler datasets like MNIST, CIFAR, faces, bedrooms, etc.
- We cannot directly compute the likelihoods of high-dimensional samples (real or generated) or compare their distributions.
- Many GAN approaches claim mainly to improve stability, which is hard to evaluate.
- Some scores to be used in the evaluation of generative models → → →

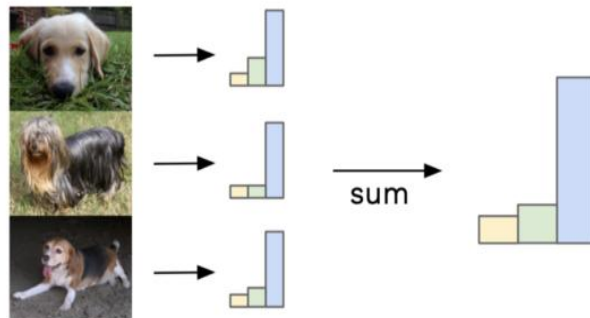


Your score on this question is 5/9

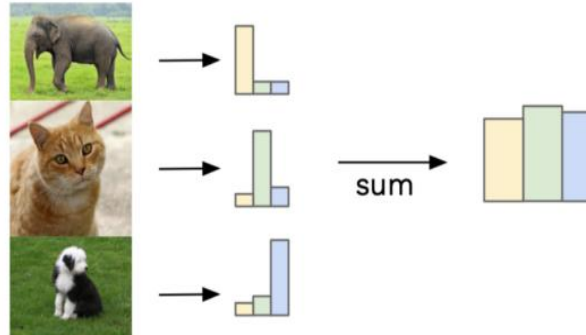
INCEPTION SCORE (IS)

- Idea: generators should produce images with a **variety** of **recognizable** object classes.
- The score is calculated based on the output of **an image classification model** (Inceptionv3)
- **IS** is maximized if:
 - The entropy of the distribution of labels for the generated images is minimized.
 - The predictions of the classification model are evenly distributed across all possible labels.

Similar labels sum to give focussed distribution



Different labels sum to give uniform distribution





INCEPTION SCORE (IS)

- Problems:
 - A GAN that simply memorizes the training data (**overfitting**) or outputs a single image per class (**mode dropping**) could still score well.

FRÉCHET INCEPTION DISTANCE

- **FID** is a metric used to evaluate the quality of images generated by GANs. It compares the *statistics of real* and *generated images* using features extracted from a pre-trained **Inception v3 network**.
- Algorithm:
 - Extract features (usually from the **pool3 layer** of Inception v3) for real and generated images.
 - Fit a multivariate Gaussian to each set of features.
 - Compute the Fréchet distance (aka Wasserstein-2 distance) between the two Gaussians.

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

μ_r, Σ_r : mean and covariance of real images' features

μ_g, Σ_g : mean and covariance of generated images' features



FRÉCHET INCEPTION DISTANCE

- **Pros:** Lower FID means generated images are **more similar** to real ones (in terms of content and diversity).
- **Cons:** Cannot detect **overfitting**.

RECALL THAT:

- Training GAN is **hard** due to several reasons mentioned before:
 - Several improvements proposed over the years, mostly changing the **loss function**.
 - **Mode Collapse**: The generator ends up modelling only a small subset of the training data.

Name	Value Function
GAN	$L_D^{GAN} = E[\log(D(x))] + E[\log(1 - D(G(z)))]$ $L_G^{GAN} = E[\log(D(G(z)))]$
LSGAN	$L_D^{LSGAN} = E[(D(x) - 1)^2] + E[D(G(z))^2]$ $L_G^{LSGAN} = E[(D(G(z)) - 1)^2]$
WGAN	$L_D^{WGAN} = E[D(x)] - E[D(G(z))]$ $L_G^{WGAN} = E[D(G(z))]$ $W_D \leftarrow clip_by_value(W_D, -0.01, 0.01)$
WGAN_GP	$L_D^{WGAN_GP} = L_D^{WGAN} + \lambda E[(\nabla D(\alpha x - (1 - \alpha G(z))) - 1)^2]$ $L_G^{WGAN_GP} = L_G^{WGAN}$
DRAGAN	$L_D^{DRAGAN} = L_D^{GAN} + \lambda E[(\nabla D(\alpha x - (1 - \alpha x_p)) - 1)^2]$ $L_G^{DRAGAN} = L_G^{GAN}$
CGAN	$L_D^{CGAN} = E[\log(D(x, c))] + E[\log(1 - D(G(z), c))]$ $L_G^{CGAN} = E[\log(D(G(z), c))]$
infoGAN	$L_{D,Q}^{infoGAN} = L_D^{GAN} - \lambda L_I(c, c')$ $L_G^{infoGAN} = L_G^{GAN} - \lambda L_I(c, c')$
ACGAN	$L_{D,Q}^{ACGAN} = L_D^{GAN} + E[P(class = c x)] + E[P(class = c G(z))]$ $L_G^{ACGAN} = L_G^{GAN} + E[P(class = c G(z))]$
EBGAN	$L_D^{EBGAN} = D_{AE}(x) + \max(0, m - D_{AE}(G(z)))$ $L_G^{EBGAN} = D_{AE}(G(z)) + \lambda \cdot PT$
BEGAN	$L_D^{BEGAN} = D_{AE}(x) - k_t D_{AE}(G(z))$ $L_G^{BEGAN} = D_{AE}(G(z))$ $k_{t+1} = k_t + \lambda(\gamma D_{AE}(x) - D_{AE}(G(z)))$

TRAINING TRICKS



UNIVERSITÀ
di **VERONA**

Improved Techniques for Training GANs

Tim Salimans
tim@openai.com

Ian Goodfellow
ian@openai.com

Wojciech Zaremba
woj@openai.com

Vicki Cheung
vicki@openai.com

Alec Radford
alec@openai.com

Xi Chen
peter@openai.com

Abstract

We present a variety of new architectural features and training procedures that we apply to the generative adversarial networks (GANs) framework. Using our new techniques, we achieve state-of-the-art results in semi-supervised classification on MNIST, CIFAR-10 and SVHN. The generated images are of high quality as confirmed by a visual Turing test: our model generates MNIST samples that humans cannot distinguish from real data, and CIFAR-10 samples that yield a human error rate of 21.3%. We also present ImageNet samples with unprecedented resolution and show that our methods enable the model to learn recognizable features of ImageNet classes.

1 Introduction

Generative adversarial networks [1] (GANs) are a class of methods for learning generative models based on game theory. The goal of GANs is to train a generator network $G(z; \theta^{(G)})$ that produces samples from the data distribution, $p_{\text{data}}(\mathbf{x})$, by transforming vectors of noise $\mathbf{z}$ as $\mathbf{x} = G(\mathbf{z}; \theta^{(G)})$. The training signal for G is provided by a discriminator network $D(\mathbf{x})$ that is trained to distinguish samples from the generator distribution $p_{\text{model}}(\mathbf{x})$ from real data. The generator network G in turn is then trained to fool the discriminator into accepting its outputs as being real.

Recent applications of GANs have shown that they can produce excellent samples [2, 3]. However, training GANs requires finding a Nash equilibrium of a non-convex game with continuous, high-dimensional parameters. GANs are typically trained using gradient descent techniques that are designed to find a low value of a cost function, rather than to find the Nash equilibrium of a game. When used to seek for a Nash equilibrium, these algorithms may fail to converge [4].

In this work, we introduce several techniques intended to encourage convergence of the GANs game. These techniques are motivated by a heuristic understanding of the non-convergence problem. They lead to improved semi-supervised learning performance and improved sample generation. We hope that some of them may form the basis for future work, providing formal guarantees of convergence.

All code and hyperparameters may be found at <https://github.com/openai/improved-gan>.

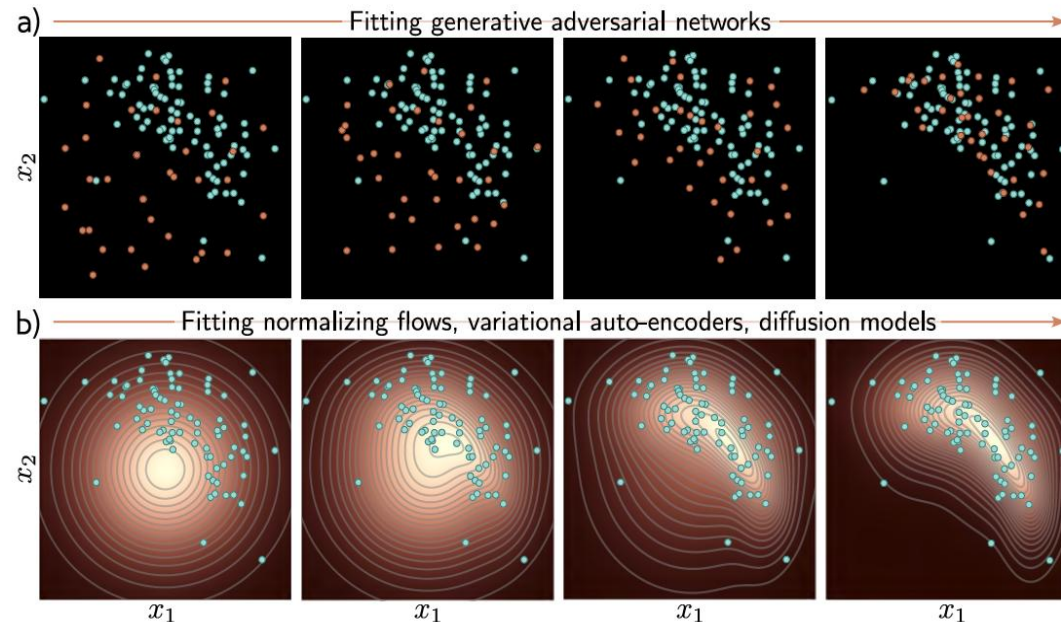
2 Related work

Several recent papers focus on improving the stability of training and the resulting perceptual quality of GAN samples [2, 3, 5, 6]. We build on some of these techniques in this work. For instance, we use some of the “DCGAN” architectural innovations proposed in Radford et al. [3], as discussed below.

One of our proposed techniques, *feature matching*, discussed in Sec. 3.1, is similar in spirit to approaches that use maximum mean discrepancy [7, 8, 9] to train generator networks [10, 11].

GANs VS VAEs

- GANs provide a mechanism for generating samples. During training, the GAN loss function encourages these samples to become progressively less distinguishable from real data.
- VAEs learn a probability distribution over the training data. This distribution can be used to draw new samples and assess the probability of new data points.



GENERATIVE MODELS PROPERTIES

- ★ **High-quality sampling:** The samples should be indistinguishable from the real data that the model was trained with.
- ★ **Coverage:** Samples should represent the entire training distribution. It is insufficient to only generate samples that all look like a subset of the training data.
- ★ **Well-behaved latent space:** Every latent variable z should correspond to a plausible data example x and smooth changes in z should correspond to smooth changes in x .
- ★ **Interpretable latent space:** Manipulating each dimension of z should correspond to changing an interpretable property of the data.
- ★ **Efficient likelihood computation:** If the model is probabilistic, we would like to be able to calculate the probability of new examples efficiently and accurately.

GANs VS VAEs



UNIVERSITÀ
di **VERONA**

	GANs	VAEs
Efficient sampling	✓	✓
High quality	✓	✗
Coverage	✗	?
Well-behaved latent space	✓	✓
Interpretable latent space	?	?
Efficient likelihood	n/a	✗



That's all

Cigdem Beyan
cigdem.beyan@univr.it
<https://cbeyan.github.io/>